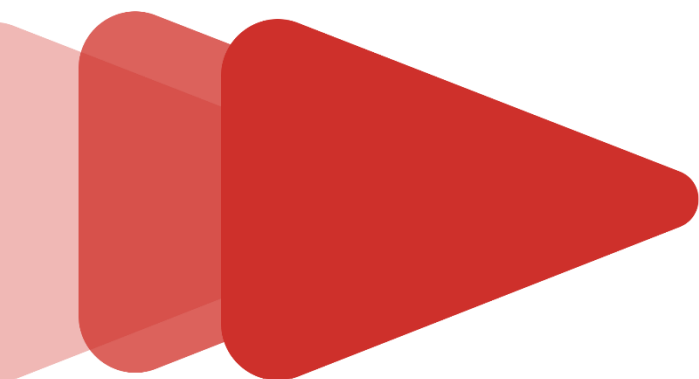




Arquitectura de la Solución Tecnológica Entrega y Recepción de Información Fiscal – Gestión Anexos (R2)

NAC-PTITARS26-00000020

***Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología***

Agosto 2026

Versión: 2.0

INFORMACIÓN DEL DOCUMENTO

Propósito:	El presente documento contiene las definiciones de arquitectura, estructura de componentes y su interacción para la Solución Tecnológica de Entrega y Recepción de Información Fiscal – Gestión Anexos, Release 2.
Alcance:	Contempla las definiciones de arquitectura de software, datos e infraestructura del Release 2 del Sistema de Gestión de Anexos Tributarios. Extiende el Intranet construido en Release 1 hacia el componente Internet para soportar, de forma genérica y parametrizable, elaboración, recepción, validación Nivel1/ Nivel2 y Nivel 3 cuando aplique, procesamiento asíncrono, consulta y seguimiento, recargas, talón resumen, cumplimiento, notificaciones, auditoría y convivencias institucionales. Declaración Patrimonial (DPT) es el primer Producto Tributario y no constituye un dominio ni microservicio específico.
Audiencia:	Dirección Nacional de Tecnología

REVISIONES

Versión/Fecha	Autor	Descripción
2.0 / 26/08/2026	Santiago Simbaña	Emisión inicial para el Release 2.

Firmas de Responsabilidad

Fecha: 26/08/2026	Dirección Nacional de Tecnología	VERSIÓN 2.0	PÁGINAS
CARGO		FIRMA	
APROBADO POR:	Ing. Johanna Chagñay C. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN-DATOS		
REVISADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		
COLABORA:	Ing. Sofia Sevilla A. EXPERTO NACIONAL DE DESARROLLO		
ELABORADO POR:	Ing. Santiago Simbaña H. EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	 Validar únicamente en FirmaBC. Firmado electrónicamente por: SANTIAGO DAVID SIMBANA HAYO	

Nota: Desarrollo colabora dentro del ámbito de sus competencias, en función del conocimiento técnico disponible en la institución. Por tanto, no interviene en la definición de las plataformas o herramientas a utilizar. En caso de que la incursión de esta nueva tecnología a nivel institucional implique ajustes o modificaciones a los aspectos contemplados en el presente documento, será necesario realizar su correspondiente actualización.

Contenido

1.	Introducción	5
2.	Arquitectura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos.....	6
2.1.	Modelo de componentes externos y su interacción	6
2.1.1.	Solución Tecnológica	6
2.1.2.	Aplicación	7
2.2.	Arquitectura de software de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos	9
2.2.1.	Definiciones de Arquitectura	9
2.2.2.	Modelo de componentes internos Ambiente Intranet	14
2.2.3.	Modelo de componentes internos Ambiente Internet	22
2.3.	Arquitectura de datos de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos	29
2.3.1.	Definiciones de Arquitectura	29
2.3.2.	Modelo de componentes internos – ambiente Intranet	30
2.3.3.	Modelo de componentes internos – ambiente Internet	31
2.4.	Arquitectura de infraestructura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos.....	32
2.4.1.	Definiciones de Arquitectura	32
2.4.2.	Modelo de Despliegue Ambiente Intranet	34
2.4.3.	Modelo de Despliegue Ambiente Internet	37
3.	Glosario	41
3.1.	Acrónimos	41
3.2.	Definiciones	42
4.	Bibliografía	44
5.	Anexos	44
5.1.	Anexo A: Versión de tecnologías	44
5.2.	Anexo B: Matriz de integraciones y convivencias	46
5.3.	Anexo C: Estructura librería pistas-auditoria-lib v1.0.0.....	46

1. Introducción

El Sistema de Gestión de Anexos (SGA) constituye el componente institucional para la administración y presentación de anexos tributarios del Servicio de Rentas Internas. El Release 1 (Intranet) estableció las capacidades de diseño, estructura, catálogos y habilitación de anexos, operadas por el funcionario institucional del SRI, con convivencia sobre los componentes legados desplegados en JBoss EAP 8.

El Release 2 evoluciona el dominio de "estructura del anexo" hacia "Gestión de Anexos Tributarios" en su alcance completo: incorpora el canal Internet (SRI en Línea), el primer Producto Tributario — la Declaración Patrimonial Tributaria (DPT), y separa explícitamente Intranet (gobierno y publicación de configuración) de un Internet (ejecución transaccional del ciclo de vida del anexo). Esta separación evita convertir la DPT en un conjunto de microservicios específicos: los componentes se diseñan a nivel de dominio genérico, reutilizables para futuros Productos Tributarios.

La arquitectura se sustenta en los siguientes principios institucionales y de diseño:

- Domain-Driven Design (DDD) estratégico y táctico.
- Arquitectura Hexagonal / Clean, con puertos y adaptadores que aíslan el dominio de la infraestructura.
- Arquitectura orientada a eventos (EDA) sobre Kafka, con patrones Transactional Outbox / Inbox e idempotencia para desacoplar la Recepción (síncrona) del Procesamiento (asíncrono).
- CQRS selectivo para las consultas y el seguimiento, con read models derivados de eventos.
- Seguridad Zero Trust: SSO/Keycloak (MFA, OIDC), autorización por backend, WSO2 API Manager como gateway de canal, WAF/F5 en el borde de Internet.
- Despliegue en Kubernetes (VKS on-premise) con Istio como service mesh, y un pipeline DevSecOps sobre GitLab (build, test, scan, deploy, GitOps).

A continuación, se detalla la lista de la documentación de la solución tecnológica con sus respectivos repositorios :

DESCRIPCIÓN	ruta
Documentación de la arquitectura de la solución tecnológica.	Repositorio: ALFRESCO Ejemplo: Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 2. ARQUITECTURA TECNOLÓGICA DE APLICACIÓN, DATOS, INFRAESTRUCTURA> ARQUITECTURA DE SOLUCIÓN TECNOLÓGICA> 02 CADENA DE VALOR> 02 CONTROL AL CUMPLIMIENTO TRIBUTARIO> ENTREGA Y RECEPCIÓN DE INFORMACIÓN FISCAL> ANEXOS>
Diagramas/modelos de la arquitectura de la solución tecnológica Gestión Anexos	Repositorio: TEAMWORK SERVER Categoría: Arquitectura de la Solución Ubicación: Data/CADENA DE VALOR/Control al Cumplimiento Tributario/Entrega y Recepción de Información Fiscal/ Gestión Anexos
Requerimiento de Usuario de la aplicación Gestión Anexos	Repositorio: ALFRESCO Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 2. ARQUITECTURA TECNOLÓGICA DE APLICACIÓN, DATOS, INFRAESTRUCTURA> ARQUITECTURA DE SOLUCIÓN TECNOLÓGICA> 02 CADENA DE VALOR> 02 CONTROL AL CUMPLIMIENTO TRIBUTARIO> ENTREGA Y RECEPCIÓN DE INFORMACIÓN FISCAL> ANEXOS> CONSTRUCCION> EDITABLES>ARS_26020_Anexos_vE2.0.0

2. Arquitectura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos

2.1. Modelo de componentes externos y su interacción

2.1.1. Solución Tecnológica

Enlace del modelo UML y descripción de los Componentes Externos de la Solución tecnológica Entrega y Recepción de información Fiscal – Gestión Anexos.

Url confluence:

<http://sriargic01:1100/pages/viewpage.action?pageId=121208936> .

2.1.2. Aplicación

La aplicación tecnológica de Gestión Anexos se ha concebido como una aplicación que está disponible para su utilización desde los ambientes internet e intranet, cuyas principales interacciones pueden darse considerando las integraciones que se muestran de manera gráfica de los componentes externos de la aplicación Entrega y Recepción de Información Fiscal.

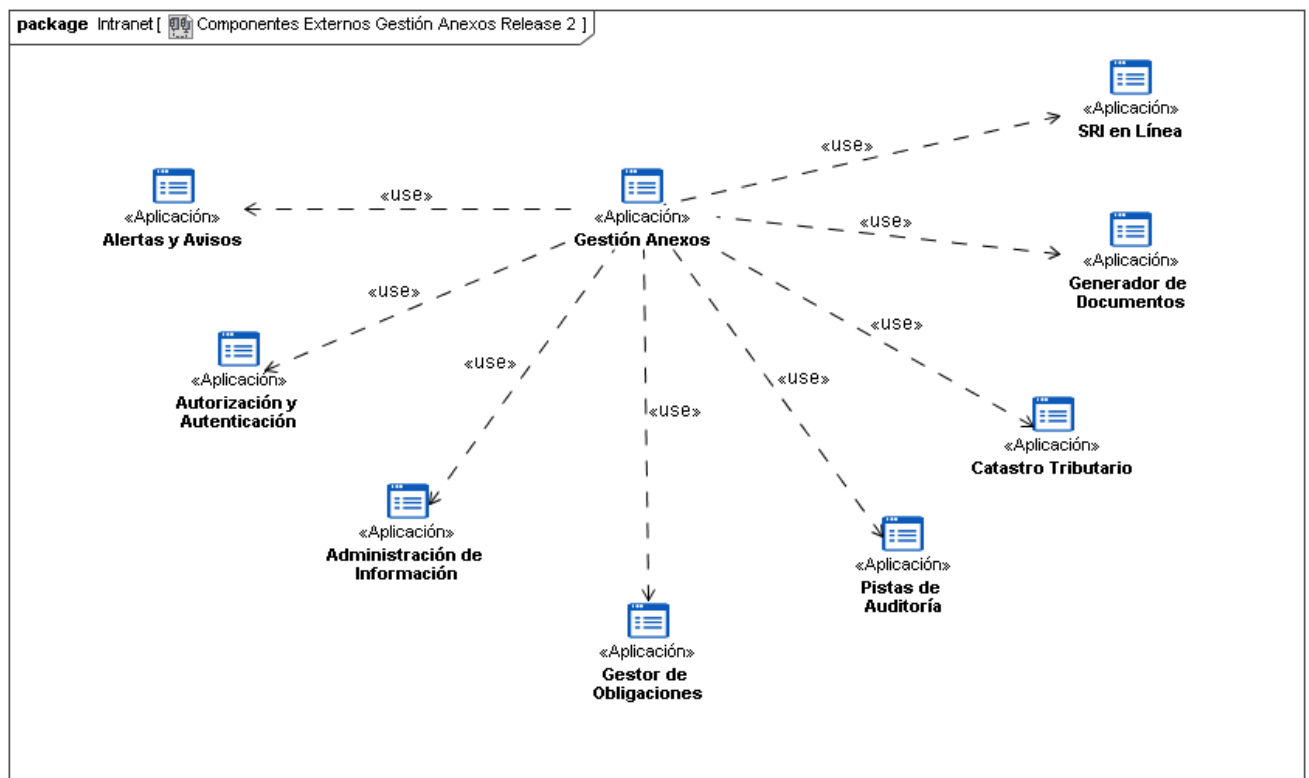


Figura 1: Modelo de Componentes Externos Entrega y Recepción de Información Fiscal - Anexos

Aplicación	Relación
Gestión Anexos	La aplicación Anexos constituye el núcleo funcional encargado de la gestión integral del ciclo de vida de los anexos tributarios en Internet e Intranet. Coordina la interacción con componentes transversales institucionales para obtener catálogos, contribuyentes, registrar el cumplimiento de las obligaciones, relacionada al tipo operativo del anexo, el registro de la pista de auditoría respectiva y emitir notificaciones. Las integraciones se realizan mediante servicios REST gestionados por el API Management, autenticados por el RedHat SSO(Keycloak) y operados por los microservicios `sri-anexos-*service`. La Aplicación Intranet en el primer Release permite la gestión interna de las plantillas de anexos, su configuración y ciclo de aprobación aprovisionando los servicios a ser utilizados en el release 2 y posteriores, desde su componente en internet que

	permite la elaboración, recepción, validación y procesamiento.
Administración de Información	Proporciona los catálogos institucionales utilizados por la solución Anexos para el uso de parámetros, tipos de impuestos, y metadatos transversales. La interacción se realiza por servicios REST que actúan como fuente única de datos maestros, garantizando consistencia y gobernanza institucional.
Autenticación y Autorización	Servicio centralizado basado en Red Hat SSO/Keycloak, utilizado para la validación de accesos y permisos mediante tokens OAuth2/OpenID. El sistema Anexos valida cada solicitud, obtiene roles autorizados y garantiza la seguridad en los flujos tanto internos como externos.
Alertas y Avisos	Permite emitir notificaciones automáticas conforme la configuración realizada de las plantillas en Alertas y Avisos hacia un grupo de funcionarios a través de correo institucional. Así como la emisión de notificaciones automáticas mediante correo y/o buzón, para notificar el estado del Anexo registrado por el contribuyente El sistema gestión Anexos se integra para la emisión de correos para informar el estado del ciclo de aprobación de la plantilla (solicitud de revisión, aceptado, devuelto con observaciones); así como el estado en el ciclo de vida del Anexo. La integración asegura trazabilidad y comunicación oportuna.
Catastro Contributivo	Repositorio oficial de datos del sujeto pasivo. El sistema Gestión Anexos consulta información del contribuyente (estado del RUC, razón social, tipo de contribuyente) para validar que el anexo sea presentado por el sujeto catastrado en la administración tributaria.
Gestor de Obligaciones	Provisiona el mecanismo para el registro del cumplimiento de las obligaciones tributarias. El sistema Gestión Anexos comunicará en su momento, realizado el procesamiento exitoso de un anexo, la necesidad de que la obligación relacionada al Anexo como cumplida.
Pistas de Auditoria	El mecanismo transversal institucional para el registro de eventos auditables. El sistema Gestión Anexos registra eventos como autenticación, y cambios de estado, en función del requerimiento funcional con el objetivo de dar cumplimiento a normativas de seguridad y trazabilidad.
Generador de Documentos	Provisiona el mecanismo de generar y gestionar documentos de forma transaccional, manual o masiva. El sistema Gestión de Anexos lo utiliza en la integración de Alertas y Avisos para la notificación vía correo electrónico utilizando las plantillas.
SRI en línea (Portal Internet)	Portal transaccional institucional donde se aloja “sri-gestion-anexos-web-internet”; el contribuyente se autentica vía SSO/Keycloak (MFA, OIDC) y accede a la elaboración, envío y consulta de anexos.

2.2. Arquitectura de software de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos

2.2.1. Definiciones de Arquitectura

Definición: Utilizar un JDK compatible con OpenJDK, conforme a la versión establecida en el estándar institucional vigente. Para el presente release, se adopta GraalVM Community Edition como distribución del JDK para la ejecución de los servicios, garantizando el cumplimiento del estándar Java definido

Motivación: Se asegura la compatibilidad, estabilidad y soporte institucional en los entornos de ejecución, así como el cumplimiento de los lineamientos corporativos de seguridad, mantenimiento y arquitectura resiliente, establecidos en el Estándar de Backend – Arquitectura resiliente. Adicionalmente, la adopción de GraalVM permite optimizar el tiempo de arranque y el consumo de recursos en entornos contenerizados, manteniendo la portabilidad, gobernanza tecnológica y alineación con el estándar Java institucional.

Definición: Implementar Quarkus como framework backend para todos los servicios sri-anexos-*--service y adapters institucionales.

Motivación: Quarkus ofrece tiempos de arranque optimizados, menor huella de memoria y soporte nativo para entornos contenerizados, lo que mejora el rendimiento y escalabilidad de la arquitectura de microservicios.

Definición: Utilizar Vue.js como framework de frontend para el portal Intranet del Sistema Gestión Anexos.

Motivación: Proporciona un modelo reactivo, alta productividad en el desarrollo y una experiencia de usuario moderna, alineada con los lineamientos institucionales de interfaces web.

Definición: Implementar servicios REST para la exposición e integración de funcionalidades del dominio Anexos.

Motivación: Facilita la interoperabilidad entre equipos y sistemas heterogéneos mediante HTTP estándar, simplificando consumo, prueba, documentación y mantenimiento.

Definición: Definir y documentar todas las APIs RESTful con OpenAPI antes de su implementación.

Motivación: Establece contratos claros desde etapas tempranas, permite generación automática de artefactos (mocking, SDKs, pruebas) y asegura gobernanza de APIs bajo el api management.

Definición: Utilizar RH-SSO/Keycloak como proveedor único de identidad para autenticación en los módulos de Intranet e Internet.

Motivación: Centraliza la gestión de sesiones, roles y permisos, reduce vulnerabilidades y garantiza cumplimiento de estándares institucionales de seguridad.

Definición: Contenerizar todos los componentes (frontend, backend, adapters, workers/event listeners) y desplegarlos en la Plataforma de Contenedores.

Motivación: Aporta portabilidad, consistencia entre ambientes, autoscaling, monitoreo nativo y cumplimiento de la estrategia institucional de orquestación.

Definición: Todos los microservicios transaccionales del dominio Anexos deberán utilizar Oracle como motor de base de datos autorizado.

Motivación: Garantiza integridad, soporte institucional, alta disponibilidad y alineación con la arquitectura de datos institucional.

Definición: Se define el uso de Oracle Database 19c como base de datos relacional obligatoria para los componentes transaccionales de la solución Gestión Anexos. Para las estructuras de información semiestructurada en formato JSON, considerando las capacidades disponibles en Oracle Database 19c, su almacenamiento deberá realizarse mediante tipos de datos soportados como CLOB, BLOB o VARCHAR2, según las características y volumen de la información, aplicando restricciones de validación IS JSON, funciones SQL/JSON y mecanismos de indexación cuando corresponda para garantizar la integridad, consulta y eficiencia en el acceso a los datos.

Motivación: Permite consolidar la persistencia de la solución sobre Oracle Database 19c como plataforma de base de datos definida arquitectónicamente para el Release 2, aprovechando sus capacidades para gestionar de manera integrada información relacional y estructuras JSON. El modelo combina persistencia relacional y semiestructurada manteniendo transaccionalidad ACID, integridad y capacidades de consulta, sin introducir un motor documental adicional. Esta decisión refuerza la mantenibilidad, eficiencia y alineación con estándares modernos de almacenamiento híbrido (relacional + semiestructurado).

Definición: Toda comunicación e integración con sistemas legados institucionales se realizará mediante servicios web REST de integración definidos para tal fin, los cuales serán consumidos por los microservicios. Los microservicios no deberán acceder directamente a bases de datos, artefactos, componentes internos o mecanismos propietarios de los sistemas legados, salvo excepción técnica formalmente aprobada.

Motivación: Esta decisión desacopla la solución Gestión Anexos de los sistemas legados, estandariza la integración, facilita monitoreo y auditoría, y reduce riesgos por dependencia tecnológica directa.

Definición: Todas las APIs expuestas deberán ser gestionadas y publicadas a través del API Management, el cual actuará como punto de entrada controlado para los consumidores.

Motivación: Permite una exposición segura, gobernada, monitoreable y alineada con las políticas institucionales, manteniendo además una arquitectura desacoplada entre la capa de gestión de APIs y la plataforma de ejecución de microservicios.

Definición: La solución Gestión Anexos aplicará los principios de Domain-Driven Design —DDD— para delimitar sus capacidades funcionales mediante bounded contexts, los cuales servirán como base para la definición de microservicios, modelos de dominio, entidades, objetos de valor, agregados y servicios de dominio, según corresponda a la complejidad funcional de cada componente.

Los eventos de dominio o integración deberán definirse únicamente cuando exista una necesidad real de comunicación desacoplada, trazabilidad funcional o propagación de cambios entre contextos

Motivación: Permite establecer límites funcionales claros, reducir acoplamiento entre componentes, mantener coherencia en el modelo de dominio y facilitar la evolución controlada. Asimismo, evita que los microservicios se definan únicamente por criterios técnicos o de operación, asegurando que su diseño responda a capacidades de negocio, responsabilidades bien delimitadas y reglas funcionales propias del dominio.

Definición: Todos los microservicios deberán integrar logging centralizado (EFK/ELK), métricas y trazabilidad distribuida.

Motivación: Permite diagnóstico, auditoría, supervisión activa y cumplimiento de normativas internas.

Definición: La solución deberá diseñarse sobre el dominio Gestión de Anexos Tributarios. La Declaración Patrimonial constituye el primer Producto Tributario implementado sobre la plataforma y no deberá convertirse en el dominio tecnológico de la solución.

Motivación: Permite desarrollar capacidades reutilizables para futuros anexos tributarios y evita que la arquitectura quede acoplada a un único producto, formulario o requerimiento normativo.

Definición: No se deberán implementar microservicios, APIs, componentes, tablas, flujos o reglas de propósito general cuyo diseño dependa exclusivamente de la Declaración Patrimonial. Las particularidades de un Producto Tributario deberán resolverse mediante estructura, metadata, parametrización, catálogos, reglas, versiones y artefactos publicados.

Motivación: Evita hardcode funcional, reduce cambios de software ante modificaciones normativas y permite incorporar nuevos productos tributarios sin duplicar la arquitectura.

Definición: Los componentes Quarkus deberán conservar el modelo reactivo/no bloqueante cuando el flujo implique operaciones de entrada/salida y deberán evitar bloquear innecesariamente los hilos de ejecución reactivos.

Motivación: Favorece utilización eficiente de recursos y permite sostener mayor concurrencia en procesos de recepción, consulta e integración.

Definición: El frontend podrá ejecutar validaciones orientadas a experiencia de usuario, pero ninguna validación realizada exclusivamente en el cliente deberá considerarse control autoritativo de negocio o seguridad.

Motivación: El código ejecutado en el navegador puede ser alterado o evitado; por ello, los controles autoritativos deben permanecer en los componentes backend correspondientes.

Definición: Se deberá utilizar Event-Driven Architecture —EDA— y Apache Kafka cuando exista necesidad de desacoplamiento temporal, procesamiento asíncrono, propagación de eventos o independencia operacional entre capacidades. REST deberá permanecer como mecanismo de comunicación para interacciones request-response donde la respuesta inmediata sea necesaria.

Motivación: Permite aplicar asincronía únicamente donde aporta resiliencia, desacoplamiento o capacidad de procesamiento, evitando introducir complejidad distribuida sin una necesidad arquitectónica.

Definición: Event Sourcing no deberá utilizarse como estrategia general de persistencia de la solución.

Motivación: Las necesidades actuales de trazabilidad, procesamiento y consulta pueden resolverse mediante persistencia transaccional, eventos de integración, auditoría y CQRS selectivo, sin introducir la complejidad operacional y de reconstrucción propia de Event Sourcing.

Definición: Los componentes que deban persistir un cambio transaccional y posteriormente publicar un evento deberán implementar Transactional Outbox o un mecanismo equivalente aprobado que mantenga consistencia entre el estado persistido y el evento a publicar.

Motivación: Evita escenarios donde la transacción de negocio se confirme, pero el evento se pierda, o donde se publique un evento sobre un cambio que finalmente no fue persistido.

Definición: Los consumidores de eventos que produzcan efectos de negocio deberán implementar Inbox / Idempotent Consumer o un mecanismo equivalente de deduplicación.

Motivación: Kafka proporciona entrega al menos una vez en determinados escenarios; por tanto, la solución debe tolerar reentregas sin duplicar efectos funcionales.

Definición: Las operaciones expuestas que puedan ser reintentadas por el consumidor y cuyo procesamiento produzca efectos persistentes deberán considerar idempotencia. La recepción de cargas deberá utilizar una clave de idempotencia y validar además el fingerprint de la solicitud.

Motivación: Evita crear múltiples recepciones ante reintentos HTTP, problemas de red o repetición involuntaria de una misma operación.

Definición: Los errores técnicos transitorios deberán manejarse mediante Retry con Backoff y, cuando corresponda, Circuit Breaker. Una vez agotada la política de reintentos, el mensaje podrá ser enviado a DLQ o marcado para suspensión técnica conforme a la política operacional aprobada.

Motivación: Permite recuperación automática ante fallos temporales sin confundirlos con errores funcionales.

Definición: La DLQ deberá considerarse un mecanismo operacional de recuperación y no un estado tributario del anexo.

Motivación: Separa las necesidades de operación técnica de la semántica funcional presentada al contribuyente.

Definición: Todo mecanismo de replay deberá conservar idempotencia, trazabilidad y auditoría, evitando producir nuevamente efectos funcionales ya completados.

Motivación: Permite recuperar procesamiento fallido sin duplicar presentaciones, obligaciones, notificaciones, talones u otros efectos.

Definición: Los Read Models podrán mantener consistencia eventual y deberán construirse mediante proyecciones derivadas de los eventos o mecanismos definidos por los Bounded Contexts propietarios.

Motivación: Desacopla el modelo de lectura del modelo transaccional y permite evolucionar consultas sin afectar operaciones de escritura.

Definición: Los nuevos microservicios Quarkus deberán utilizar la librería institucional sri-pistas-auditoria-lib para capturar las transacciones definidas como auditables. No se deberá crear un microservicio de negocio independiente para centralizar la auditoría del SGA.

Motivación: Reutiliza el mecanismo institucional definido, mantiene desacoplamiento mediante AuditPort y evita duplicar una capacidad transversal.

Definición: Una falla interna del mecanismo de transporte de auditoría no deberá introducir de manera indiscriminada una caída del microservicio de negocio; deberá aplicarse la estrategia de contingencia definida por la librería institucional.

Motivación: La librería fue diseñada como componente transversal no invasivo y desacoplado del procesamiento principal.

2.2.2. Modelo de componentes internos Ambiente Intranet

El modelo de componentes internos del ambiente Intranet describe los elementos técnicos que conforman el Sistema de Gestión de Anexos (Intranet), sus responsabilidades, dependencias directas y transitivas, mecanismos de integración y tecnologías utilizadas para la administración, parametrización, habilitación y publicación de los Productos Tributarios (Anexos). El modelo se sustenta en una arquitectura de microservicios orientada a dominio mediante DDD y Arquitectura Hexagonal, expuesta a través de WSO2 API Management, integrada con Red Hat SSO/Keycloak para autenticación y autorización, con persistencia en Oracle INTRASRI, esquema GESTION_ANEXOS, y uso de Apache Kafka para eventos asíncronos y pistas de auditoría, así como MinIO para el almacenamiento de artefactos publicados. Adicionalmente, incorpora los componentes transversales de auditoría, observabilidad e integraciones institucionales requeridas por la solución, manteniendo claramente diferenciadas las dependencias propias del ambiente Intranet respecto de las capacidades que serán consumidas posteriormente por el ambiente Internet.

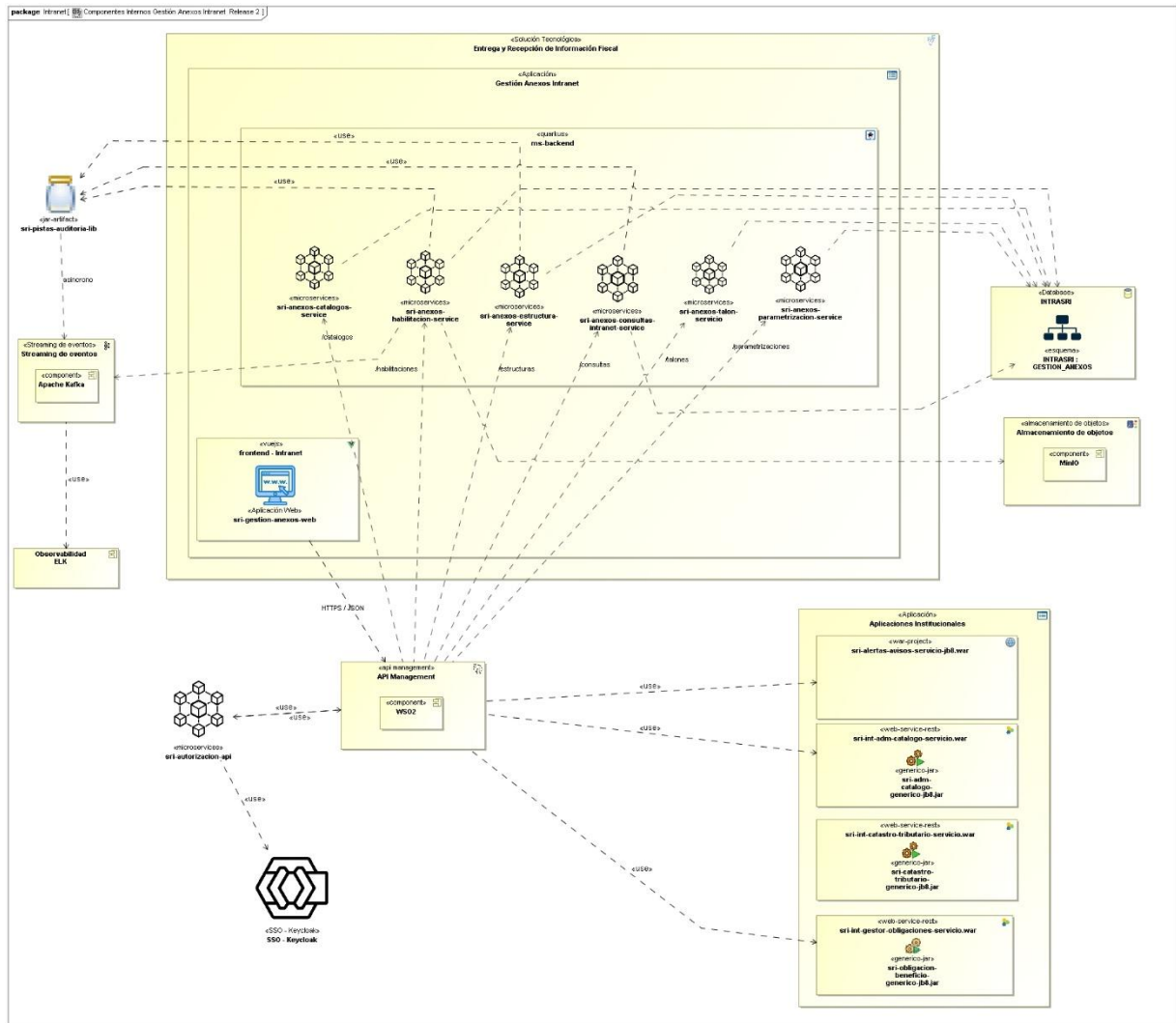


Figura 2: Modelo de Componentes Internos Intranet Entrega y Recepción de Información Fiscal – Gestión Anexos (INTRANET)

Artefacto/Microservicio	Descripción	Tecnología Utilizada
Anexos:		
sri-anexos-estructura-service	Descripción: Gestiona la estructura de las plantillas de Gestión Anexos, desde su creación y configuración inicial hasta el control de sus estados dentro del flujo de construcción y solicitud de revisión. Tipo: Microservicio de dominio Bounded Context: Estructura de Anexos(definición de anexos, secciones, casilleros, versiones y estados de diseño) Agregados principales: Anexo, Sección, Casillero, VersionEstructura, campo Endpoints base: estructuras/api/v1/ Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS.	- Java JDK versión: Anexo A. - Quarkus (REST JSON-B, Mutiny): Anexo A. - Lombok: Anexo A - Hibernate Reactive - Panache Reactive - OIDC/Keycloak - MapStruct: Anexo A - OpenAPI - Bean Validation - JUnit + RestAssured

	Dependencias directas: API Management, Oracle INTRASRI Extensión R2: la estructura agrega funcionalidad para producir información versionada.	• JaCoCo • Sonar/PMD
sri-anexos-catalogos-service	Descripción: Gestiona la creación y mantenimiento de los catálogos requeridos por la solución, incluyendo catálogos específicos, campos de información, operativos y mensajes. Estos elementos constituyen insumos base para la definición y construcción de las plantillas de Anexos dentro del proceso de Gestión de Anexos. Tipo: Microservicio de dominio Bounded Context: Catálogos de Anexos. Agregados principales: CatalogoAnexo, ValorCatalogoAnexo. Endpoints base: catalogos/api/v1/. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS Dependencias directas: API Management, Oracle INTRASRI. Dependencias transitivas: Servicios de Anexos que leen estos catálogos (estructuras, parametrizaciones, habilitación). Extensión R2: agrega funcionalidad para importar/exportar información de los catálogos.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-habilitacion-service	Descripción: Gestiona el proceso de revisión, aprobación y publicación de las plantillas de Gestión Anexos, incluyendo el manejo de los estados Devolver y Aprobar, así como el registro de las observaciones realizadas por el usuario responsable. Tipo: Microservicio de dominio Bounded Context: Habilitación de Anexos. Agregados principales: HabilitacionAnexo, EstadoHabilitacion, ObservacionHabilitacion. Endpoints base: habilitaciones/api/v1/. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI, sri-anexos-estructura-service, sri-anexos-catalogo-service. Extensión R2: amplía la revisión/aprobación de R1 para asumir formalmente la publicación de la plantilla, genera snapshot de la Plantilla y la almacena en MinIO.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD • Kafka • SmallRye Reactive Messaging en Quarkus • EDA • MinIO
sri-anexos-parametrizacion-servicio	Descripción: Gestiona la parametrización de validaciones en los diferentes niveles y de las reglas configurables del SGA, desde condiciones de visualización de casilleros hacia una capacidad general de reglas y validaciones versionadas. Tipo: Microservicio de dominio.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A

	Bounded Context: Parametrización y Reglas de Anexos. Aggregates principales: DefinicionValidacion, VersionValidacion, AsignacionValidacion, ValidationTestCase. Endpoints base: parametrizaciones/api/v1/.	• Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-talon-servicio	Descripción: Microservicio responsable de la administración y configuración del Talón Resumen dentro del ambiente Intranet. Gestiona el diseño de la plantilla del talón, su estructura de cabecera y detalle, selección y orden de campos, parametrización de información a presentar, estados de construcción/revisión/aprobación y búsqueda de plantillas. La configuración generada debe formar parte de la definición versionada del Producto Tributario(plantillas) para ser utilizada posteriormente en la generación del talón asociado a una carga procesada exitosamente. Tipo: Microservicio de dominio Bounded Context: Talón Resumen. Aggregate principal: TalonResumen. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Endpoint base: talones/api/v1. Dependencias directas: API Management, Oracle INTRASRI, SSO/Keycloak, sri-pistas-auditoria-lib.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-consultas-intranet-servicio	Descripción: Microservicio responsable de las capacidades de consulta para usuarios internos del Sistema de Gestión Anexos. Expone las operaciones requeridas por el módulo de Consultas Intranet. Tipo: Microservicio de consulta / Read Side. Bounded Context: Consultas. Patrón: CQRS selectivo; no utiliza Event Sourcing. Persistencia / Read Models: Oracle INTRASRI, esquema GESTION_ANEXOS Endpoint base: consultas/api/v1. Dependencias directas: API Management, SSO/Keycloak, sri-pistas-auditoria-lib y fuentes/proyecciones autorizadas del dominio.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
FrontEnd Intranet		
Aplicativo Web sri-gestion-anexos-web	Aplicación web usada por usuarios internos para diseñar anexos, configurar parametrizaciones, obligaciones, gestionar. Consume APIs REST del dominio Anexos.	• Vue versión: Anexo A. • Node versión: Anexo A.

	Extensión R2: amplía la administración para soportar las nuevas capacidades de estructura versionada, funcionalidad agregada de catálogos, parametrización/reglas, talón resumen, habilitación y publicación de Productos Tributarios (Plantillas de anexos y talón resumen).	• NPM versión: Anexo A. • Vue CLI versión: Anexo A. • TypeScript versión: Anexo A. • Vite versión: Anexo A. • PrimeVue versión: Anexo A. • Primeicons versión: Anexo A. • BootStrap versión: Anexo A. • Pinia versión: Anexo A. • Axios • Keycloak-js versión: Anexo A. • Formulajs: Anexo A. • vuelidate/core: Anexo A. • vuelidate/validators: Anexo A. • jexl: Anexo A. • uuid: Anexo A. • vuelidate/core: Anexo A. • read-excel-file: Anexo A.
Autenticación / SSO		
SSO / Keycloak	Proveedor de identidad corporativo; emite tokens OAuth2/OIDC usados por API Management y microservicios para validar identidad y roles.	• SSO version: Anexo A.
sri-autorizacion-api	Descripción: Gestiona permisos y roles específicos del dominio Anexos, complementando la autenticación global de Keycloak. Tipo: Microservicio de dominio (seguridad de negocio) Bounded Context: Seguridad del dominio Anexos (autorización funcional). Agregados principales: Rol, Permisos, AsignacionPermisosUsuario. Endpoints base: autorizacion/api/v1/. Persistencia: Oracle INTRASRI. Dependencias directas: API Management, Oracle INTRASRI, Keycloak. Dependencias transitivas: Resto de microservicios sri-anexos-* consultan o usan su librería para autorización fine-grained.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A. • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A. • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
Base de datos		

INTRASRI	La aplicación de anexos utiliza la base de datos INTRASRI para guardar/modificar/consultar toda la información transaccional de esta aplicación en intranet, para lo cual utiliza el esquema denominado GESTION_ANEXOS .	• Oracle Database version: Anexo A.
Aplicaciones institucionales		
sri-int-gestor-obligaciones-servicio	Descripción: Expone servicios del Gestor de Obligaciones: consulta de obligaciones tributarias (periodicidad, vencimiento, estado) y registro de obligación cumplida cuando se presenta el anexo. Tipo: Servicio Web REST de integración / fachada Gestor de Obligaciones.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-adm-catalogo-servicio	Descripción: Provee acceso a las tablas transversales de Administración de Información (ADM Año, Mes, Países, Provincias, Cantones, Parroquias, Obligaciones, Identificaciones, etc.), con navegación jerárquica según el nivel requerido. Tipo: Servicio Web REST de integración / catálogos transversales.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-catastro-tributario-servicio	Descripción: Expone servicios de consulta de información del contribuyente (RUC, razón social, medios de contacto, atributos adicionales) a partir del sistema legado de Catastro Tributario. Tipo: Servicio Web REST de integración / fachada Catastro Tributario.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-alertas-avisos-servicio-jb8	Descripción: Servicio Web que provee operaciones para enviar notificaciones internas (correos, avisos) usando la solución institucional de Alertas y Avisos migrada a JBoss 8.x: publicación de anexo, solicitud de revisión, rechazos, etc. Tipo: Servicio Web REST de integración / mensajería de Alertas y Avisos.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A

		• Jakarta Restful Web Services: Anexo A
Genéricos		
sri-adm-catalogo-generico-jb8.jar	Componente requerido por la aplicación para la obtención de datos de catálogos transversales que corresponden a la administración del SRI, tales como: usuarios, estructura organizacional.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-catastro-tributario-generico-jb8.jar	Componente requerido por la aplicación para obtener los datos del registro único de contribuyentes esto es: número ruc, razón social, tipo y clase de contribuyente, jurisdicción, representante legal, fecha de inicio y reinicio de actividades, fecha inicio y fin de obligado a llevar contabilidad, fallecido y si es fantasma o no.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-obligacion-beneficio-generico-jb8.jar	Componente requerido por la aplicación para obtener información de consulta de obligaciones tributarias y registro de obligación.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
Librería		
sri-pistas-auditoria-lib	Descripción: Librería transversal embebida en los microservicios Quarkus para capturar y publicar pistas de auditoría sin bloquear el flujo funcional. Usa un puerto hexagonal y publica eventos hacia Kafka para su procesamiento en ELK. Tipo: JAR transversal. Patrones: Hexagonal (AuditPort + Adapter), Interceptor, Event Notification, Fallback local ante falla de transporte. Arquitectura: Hexagonal, no invasiva y no bloqueante. Dependencias directas: Microservicios Quarkus, Kafka de auditoría, OpenTelemetry.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate • Panache • OIDC/Keycloak • MapStruct: Anexo A • FluentBit (Plugin envío data a ELK) • Jakarta EE10 • OpenTelemetry • Jandex Maven Plugin • Definición Anexo C

		• Apache Kafka
Herramientas		
API Management	Descripción: Plataforma corporativa de gestión y exposición de APIs que actúa como punto controlado de acceso a los servicios del ambiente Intranet del Sistema de Gestión de Anexos. Centraliza la publicación, protección, gobierno y consumo de las APIs expuestas por los microservicios del dominio Anexos y por las integraciones institucionales que deban ser consumidas mediante interfaces síncronas. Evita que los consumidores accedan directamente a los microservicios y desacopla la ubicación e implementación interna de los servicios respecto de sus consumidores. Tipo: Plataforma de API Management / API Gateway. Dependencias directas: sri-gestion-anexos-web, microservicios e integraciones institucionales expuestos mediante APIs.	• WSO 2 API Management
Streaming de Eventos	Descripción: Plataforma de streaming y mensajería utilizada por el ambiente Intranet para la gestión asíncrona de eventos generados durante la gobernabilidad de los Productos Tributarios (Plantillas). Su principal responsabilidad en este ambiente es desacoplar el proceso de publicación realizado por sri-anexos-habilitacion-service, permitiendo identificar la disponibilidad de una nueva versión mediante un evento, sin generar una dependencia síncrona entre ambos planos. Adicionalmente, constituye el mecanismo de transporte de las pistas de auditoría generadas. Tipo: Plataforma de streaming / mensajería orientada a eventos. Patrones: Event-Driven Architecture, Publish-Subscribe, Event Notification, Message Envelope, Correlation ID. Dependencias directas: sri-anexos-habilitacion-service, sri-pistas-auditoria-lib y consumidores autorizados. Regla arquitectónica: Kafka transporta eventos, identificadores, metadata y referencias; no debe utilizarse como repositorio de los artefactos del Producto Tributario.	• Apache Kafka
Almacenamiento de Objetos	Descripción: Plataforma de almacenamiento de objetos utilizada por el ambiente Intranet para persistir los artefactos generados en el proceso de publicación. No almacena de forma genérica cualquier información producida por los microservicios, sino específicamente los artefactos generados en el proceso de habilitación.	• MinIO

	Tipo: Plataforma de almacenamiento de objetos compatible con S3. Dependencia directa: sri-anexos-habilitacion-service, responsable de la capacidad de Habilitación.	
Observabilidad ELK	Descripción: Plataforma institucional para recepción, transformación, persistencia, búsqueda y visualización de pistas de auditoría y logs operacionales. Mantiene correlación con trazas mediante traceld. Tipo: Plataforma de logging / auditoría / observabilidad.	• Elasticsearch. • Logstash. • Kibana

2.2.3. Modelo de componentes internos Ambiente Internet

El modelo de componentes internos del ambiente Internet describe los elementos técnicos que conforman el Sistema Gestión Anexos, sus responsabilidades y las relaciones y dependencias directas y transitivas existentes entre ellos. El modelo representa el flujo de atención al contribuyente desde SRI en Línea hacia la aplicación web y los servicios de elaboración, recepción, procesamiento y consulta, así como su interacción con los componentes institucionales de API Management, autenticación y autorización, procesamiento asíncrono y distribuido, persistencia, almacenamiento de objetos, integraciones institucionales, auditoría y observabilidad. Asimismo, identifica las tecnologías y mecanismos de comunicación utilizados entre los componentes, permitiendo visualizar de manera integral cómo se soporta en el ambiente Internet el flujo de recepción y procesamiento de información fiscal.

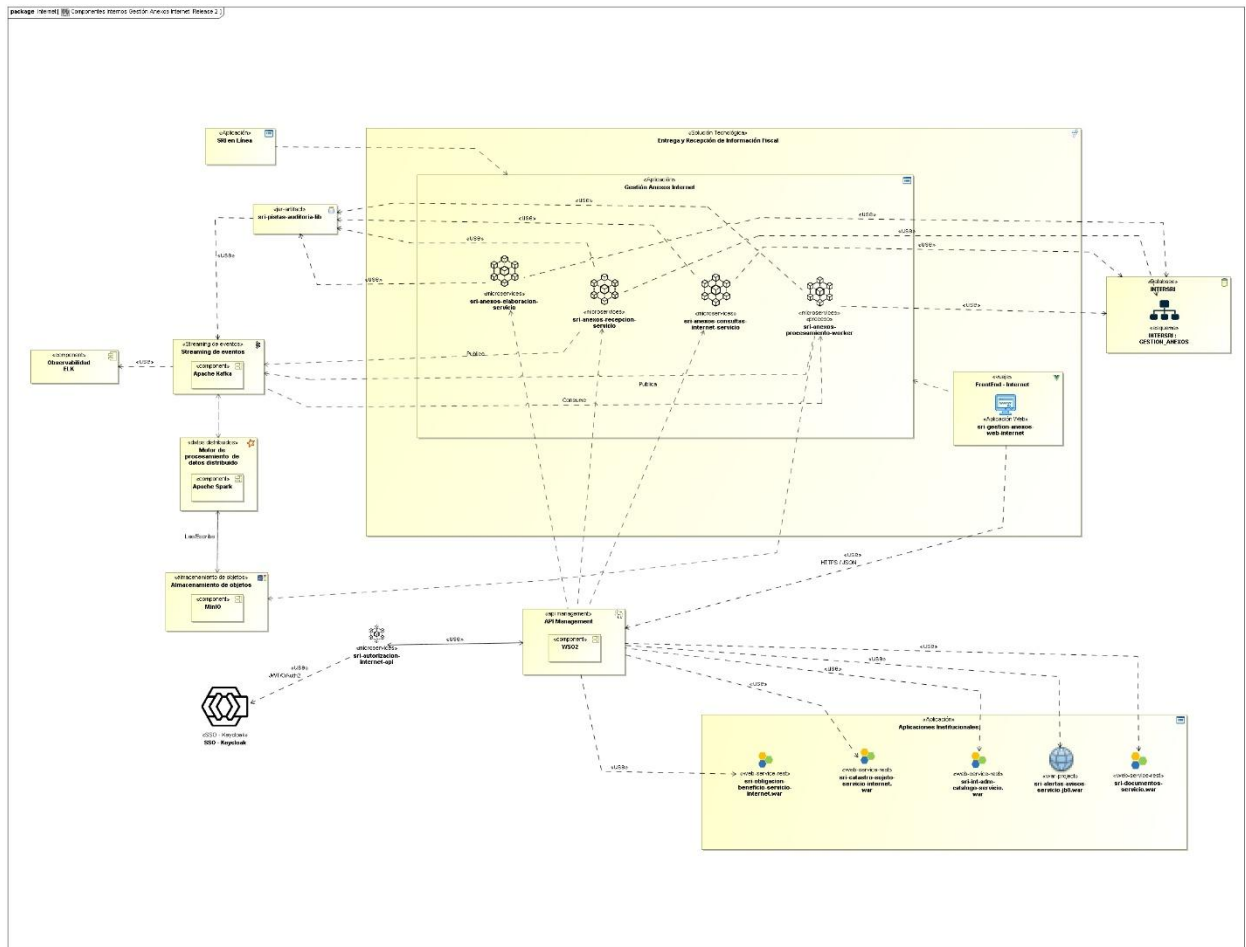


Figura 3. Modelo de Componentes Internos Internet Entrega y Recepción de Información Fiscal – Gestión Anexos (INTERNET)

Artefacto/Microservicio	Descripción	Tecnología Utilizada
Anexos:		
sri-anexos-elaboracion-service	Descripción: Microservicio responsable de la elaboración de la declaración/anexo por parte del contribuyente. Permite armar la información utilizando catálogos, configuraciones y plantillas previamente publicadas desde Gestión anexos - Intranet; persiste la información en la base de datos, no ejecuta procesamiento asíncrono ni validaciones complejas posteriores a la recepción. Tipo: Microservicio de dominio. Bounded Context: Elaboración de Anexos. Agregados principales: Endpoint base: /elaboracion/api/v1. Persistencia: Oracle INTERSRI Patrones: Repository, Application Service, Domain Service, Specification; Request-Response para operaciones interactivas; Cache-Aside para catálogos/configuración cuando aplique. Arquitectura: DDD + Hexagonal, API First, Reactiva	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD • OpenTelemetry

	Dependencias directas: WSO2 API Management, configuraciones publicadas desde Gestión Anexos – Intranet.	Quarkus Cache + Caffeine
sri-anexos-recepcion-service	Descripción: Microservicio que ejecuta la recepción inmediata del envío. Valida la estructura mínima, registra el ingreso, persiste metadatos transaccionales, almacena el objeto recibido en MinIO y publica eventos para continuar el procesamiento de forma asíncrona, sin esperar a que termine el procesamiento completo. Tipo: Microservicio de dominio. Bounded Context: Recepción de Anexos. Agregados principales: Recepción/Carga Endpoint base: /recepcion/api/v1 Persistencia: Oracle INTERSRI, MinIO. Patrones: Request-Response para recepción, Publish-Subscribe / Event Notification, Transactional Outbox para publicación confiable, Idempotency Key / Idempotent Producer, Repository, Specification para validaciones mínimas. Arquitectura: DDD + Hexagonal, EDA procesamiento reactivo/no bloqueante. Dependencias directas: WSO2 API Management, Kafka, MinIO.	- Java JDK versión: Anexo A. - Quarkus (REST JSON-B, Mutiny): Anexo A. - Lombok: Anexo A - Hibernate Reactive - Panache Reactive - OIDC/Keycloak - MapStruct: Anexo A - OpenAPI - Bean Validation - JUnit + RestAssured - JaCoCo - Sonar/PMD - SmallRye Reactive Messaging / Kafka - MinIO Client - OpenTelemetry
sri-anexos-procesamiento-worker	Descripción: Worker interno sin API pública. Consume eventos de Kafka, orquesta el flujo de procesamiento, delega cómputo distribuido en Apache Spark cuando la carga lo requiere, consolida resultados y publica eventos de salida. No es invocado directamente por el frontend ni por consumidores externos. Tipo: Worker / consumidor asíncrono de dominio. Bounded Context: Procesamiento de Anexos. Agregados principales: Ejecución/Proceso y Resultado de Procesamiento Endpoint base: No expone API. Persistencia: Oracle INTERSRI, MinIO de resultados procesados, según el flujo. Patrones: Event-Driven Consumer, Publish-Subscribe, Competing Consumers, Idempotent Consumer, Inbox Pattern, Retry Queue, DLQ, Correlation ID, Message Envelope, Saga/orquestación técnica del pipeline cuando exista coordinación multi-etapa, Backpressure. Arquitectura: EDA, DDD + Hexagonal, Reactiva, procesamiento distribuido delegado a Spark. Dependencias directas: Kafka, Apache Spark, MinIO.	- Java JDK versión: Anexo A. - Quarkus (REST JSON-B, Mutiny): Anexo A. - Lombok: Anexo A - Hibernate Reactive - Panache Reactive - OIDC/Keycloak - MapStruct: Anexo A - OpenAPI - Bean Validation - JUnit + RestAssured - JaCoCo - Sonar/PMD - SmallRye Reactive Messaging / Kafka - MinIO Client - OpenTelemetry - Cliente/Adapter Apache Spark
sri-anexos-consultas-internet-service	Descripción: Microservicio de solo lectura que expone el estado consolidado y los resultados disponibles para consulta. No procesa archivos y	Java JDK versión: Anexo A.

	no ejecuta validaciones de recepción ni procesamiento. Tipo: Microservicio de consulta. Bounded Context: Consultas Agregados principales: Consultas/Lectura Endpoint base: /consultas/api/v1. Persistencia: Oracle INTERSRI (modo lectura) resultados consolidados. Patrones: CQRS - lado Query, Read Model / Projection, Repository de lectura, API Composition cuando deba componer información ya consolidada; Cache-Aside si se justifica por demanda. Arquitectura: DDD + Hexagonal, API First, lectura reactiva. Dependencias directas: WSO2 API Management.	• Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD • OpenTelemetry
FrontEnd Internet		
sri-gestion-anexos-web-internet	Descripción: Aplicación web del ambiente Internet que presenta al contribuyente las funcionalidades de elaboración, envío y consulta de anexos. No contiene lógica de procesamiento masivo, consume capacidades backend a través de API Management. Tipo: Frontend SPA. Arquitectura: Arquitectura frontend por componentes, separación presentación/servicios, consumo API First. Patrones: SPA, Adapter de cliente HTTP, manejo centralizado de sesión/estado según estándar institucional. Dependencias directas: SRI en Línea, WSO2 API Management.	• Vue versión: Anexo A. • Node versión: Anexo A. • NPM versión: Anexo A. • Vue Cli versión: Anexo A. • TypeScript versión: Anexo A. • Vite versión: Anexo A. • PrimeVue versión: Anexo A. • Primeicons versión: Anexo A. • Bootstrap versión: Anexo A. • Pinia versión: Anexo A. • Axios • Keycloak-js versión: Anexo A. • Formulajs: Anexo A. • vuelidate/core: Anexo A. • vuelidate/validators: Anexo A. • jexl: Anexo A. • uuid: Anexo A. • vuedraggable: Anexo A. • read-excel-file: Anexo A.

SRI en Línea	Descripción: Portal institucional que habilita el acceso del contribuyente a la aplicación sri-gestion-anexos-web-internet y al resto de servicios electrónicos publicados para Internet.	- Portal transaccional institucional SRI en Línea. - HTTPS.
Autenticación / SSO		
SSO / Keycloak	Proveedor de identidad corporativo; emite tokens OAuth2/OIDC usados por API Management y microservicios para validar identidad y roles.	SSO version: Anexo A.
sri-autorizacion-internet-api	Descripción: Gestiona permisos y roles específicos del dominio Anexos, complementando la autenticación global de Keycloak. Tipo: Microservicio de dominio (seguridad de negocio) Bounded Context: Seguridad del dominio Anexos (autorización funcional). Agregados principales: Rol, Permisos, AsignacionPermisosUsuario. Endpoints base: /autorización-internet/api/v1/. Persistencia: Oracle INTERSRI. Dependencias directas: API Management, Oracle INTERSRI, Keycloak. Dependencias transitivas: Resto de microservicios sri-anexos-* consultan o usan su librería para autorización fine-grained.	- Java JDK versión: Anexo A. - Quarkus (REST JSON-B, Mutiny): Anexo A. - Lombok: Anexo A - Hibernate Reactive - Panache Reactive - OIDC/Keycloak - MapStruct: Anexo A - OpenAPI - Bean Validation - JUnit + RestAssured - JaCoCo - Sonar/PMD
Base de datos		
INTERSRI	Descripción: La aplicación de anexos utiliza la base de datos INTERSRI para guardar/modificar/consultar toda la información transaccional de esta aplicación en internet, para lo cual utiliza el esquema denominado GESTION_ANEXOS. Repositorio transaccional del ambiente Internet que almacena metadatos, estados y resultados consolidados requeridos por los microservicios del Gestor Anexos – Internet. Tipo: Base de datos transaccional. Uso: Persistencia de recepción, estados de procesamiento, información transaccional y proyecciones consultables, conforme se defina el modelo físico.	Oracle Database versión: Anexo A.
Aplicaciones institucionales		
sri- obligacion-beneficio-servicio-internet	Descripción: Expone servicios del Gestor de Obligaciones: consulta de obligaciones tributarias (periodicidad, vencimiento, estado) y registro de obligación cumplida cuando se presenta el anexo. Tipo: Servicio Web REST	- Java JDK 1.8 - Jakarta Enterprise Beans
sri-int-adm-catalogo-servicio	Descripción: Provee acceso a las tablas transversales de Administración de Información (ADM Año, Mes, Países, Provincias, Cantones, Parroquias, Obligaciones, Identificaciones, etc.), con navegación jerárquica según el nivel requerido.	- Java JDK versión: Anexo A. - Jakarta Enterprise Beans: Anexo A

	Tipo: Servicio Web REST	• Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-catastro-sujeto-servicio-internet	Descripción: Expone servicios de consulta de información del contribuyente (RUC, razón social, medios de contacto, atributos adicionales). Tipo: Servicio Web REST	• Java JDK 1.8 • Jakarta Enterprise Beans
sri-alertas-avisos-servicio-jb8	Descripción: Servicio Web que provee operaciones para enviar notificaciones internas (correos, avisos) usando la solución institucional de Alertas y Avisos migrada a JBoss 8.x: publicación de anexo, solicitud de revisión, rechazos, etc. Tipo: Servicio Web REST	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-documentos-servicio	Servicio REST de consulta del buzón del contribuyente Tipo: Servicio Web REST de integración	• Java JDK 1.8 • Jakarta Enterprise Beans
Librería		•
sri-pistas-auditoria-lib	Descripción: Librería transversal embebida en los microservicios Quarkus para capturar y publicar pistas de auditoría sin bloquear el flujo funcional. Usa un puerto hexagonal y publica eventos hacia Kafka para su procesamiento en ELK. Tipo: JAR transversal. Patrones: Hexagonal (AuditPort + Adapter), Interceptor, Event Notification, Fallback local ante falla de transporte. Arquitectura: Hexagonal, no invasiva y no bloqueante. Dependencias directas: Microservicios Quarkus, Kafka de auditoría, OpenTelemetry.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate • Panache • OIDC/Keycloak • MapStruct: Anexo A • FluentBit (Plugin envío data a ELK) • Jakarta EE10 • OpenTelemetry • Jandex Maven Plugin • Definición Anexo C • Apache Kafka
Herramientas		•
API Management	Descripción: Punto de entrada y gobierno de APIs del ambiente Internet. Centraliza autenticación, autorización, validación de acceso, rate limiting, throttling, seguridad de cabeceras y trazabilidad	• WSO 2 API Management

	de las invocaciones hacia los servicios de Gestión Anexos e integraciones institucionales. Tipo: API Management / Gateway. Patrones: API Gateway, Gateway Offloading, Proxy, Rate Limiting, Throttling. Dependencias directas: sri-autorizacion-internet-api, SSO/Keycloak, microservicios y aplicaciones institucionales.	
Almacenamiento de Objetos	Descripción: Almacenamiento de objetos utilizado para conservar los archivos/objetos recibidos por el proceso de recepción, desacoplando el contenido pesado de la persistencia transaccional. Tipo: Object Storage. Patrones: Externalized Object Storage, referencia por identificador/URI desde metadatos transaccionales. Dependencias directas: Apache Spark, sri-anexos-procesamiento-worker.	• MinIO
Observabilidad ELK	Descripción: Plataforma institucional para recepción, transformación, persistencia, búsqueda y visualización de pistas de auditoría y logs operacionales. Mantiene correlación con trazas mediante traceld. Tipo: Plataforma de logging / auditoría / observabilidad.	• Elasticsearch. • Logstash. • Kibana
Streaming Event	Descripción: Bus de eventos central del pipeline. Desacopla la recepción del procesamiento y permite que los productores continúen sin esperar la finalización de tareas posteriores. Tipo: Plataforma de streaming/mensajería. Patrones: Publish-Subscribe, Message Queue, Event Notification, Competing Consumers, Retry Queue, DLQ, Correlation ID, Message Envelope. Dependencias directas: sri-anexos-recepcion-service, sri-anexos-procesamiento-worker y productores/consumidores autorizados.	• Apache Kafka • TLS
Procesamiento Distribuido	Descripción: Motor de procesamiento distribuido utilizado por el worker para ejecutar validaciones o transformaciones de alta demanda computacional sobre cargas masivas. El worker mantiene la orquestación, Spark ejecuta el cómputo distribuido. Tipo: Motor de procesamiento distribuido. Patrones: Distributed Processing, Parallel Processing, Partitioning, Competing Workers a nivel de ejecución distribuida. Dependencias directas: sri-anexos-procesamiento-worker, MinIO para lectura/escritura cuando corresponda.	• Apache Spark • TLS

2.3. Arquitectura de datos de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos

2.3.1. Definiciones de Arquitectura

Definición: Utilizar Oracle la versión disponible institucionalmente, como mínimo 19c o superior como motor de base de datos relacional para el almacenamiento, gestión y persistencia de los datos transaccionales generados por la solución Gestión Anexos.

Motivación: Oracle Database constituye el motor relacional institucional utilizado para la gestión de información transaccional, ofreciendo capacidades de consistencia, integridad referencial, seguridad, disponibilidad, respaldo y recuperación.

Definición: La base de datos a ser utilizada por la aplicación de Gestión Anexos en Intranet es INTRASRI, así como en Internet es la base de datos INTERSRI

Motivación: Utilizar la base de datos establecida para las aplicaciones de intranet e internet con la arquitectura vigente.

Definición: Repositorio transaccional primario en el entorno Intranet del SRI. Almacena toda la información operativa del ciclo de vida de los anexos: creación, edición, validación, auditoría, estados y control de usuarios. Toda escritura ocurre exclusivamente en esta base.

Motivación: Garantiza la integridad referencial, seguridad, transaccionalidad ACID y trazabilidad de datos críticos, en un entorno controlado y no expuesto directamente a internet. Alineado con el principio de “fuente de verdad” operativa.

Definición: El esquema a ser utilizado por la aplicación es GESTION_ANEXOS.

Motivación: Dar cumplimiento a los estándares institucionales en cuanto al nombrado de objetos de bases de datos.

Definición: El diseño, construcción y mantenimiento de los objetos de base de datos de la solución Gestión Anexos Intranet/Internet deberá realizarse conforme a los estándares institucionales de base de datos, lineamientos de modelado, nomenclatura, seguridad, integridad, auditoría, administración de objetos, versionamiento de scripts y buenas prácticas definidas por la institución.

Motivación: Esta definición asegura que la base de datos de la solución mantenga consistencia técnica, trazabilidad, calidad, seguridad y facilidad de administración dentro del ecosistema institucional, evitando estructuras aisladas o criterios particulares que dificulten la operación, mantenimiento y evolución de la aplicación.

Definición: El almacenamiento de objetos deberá almacenar únicamente aquellos objetos cuya naturaleza corresponda a artefactos publicados, archivos o

representaciones semiestructuradas que no requieran comportamiento transaccional relacional.

Motivación: Evita utilizar almacenamiento de objetos como sustituto indiscriminado de la base de datos y mantiene una separación clara entre datos transaccionales y objetos publicados.

Definición: Oracle y almacenamiento de objetos tendrán responsabilidades diferentes y complementarias: Oracle almacenará estado transaccional y metadata estructurada, mientras que almacenamiento de objetos almacenará artefactos y objetos publicados.

Motivación: La separación permite utilizar cada tecnología de acuerdo con la naturaleza de los datos y evita almacenar grandes artefactos o archivos como estructuras transaccionales cuando no exista necesidad funcional para ello.

Definición: Durante la elaboración del formato, el contenido JSON que constituye su modelo transaccional podrá almacenarse y gestionarse en Oracle. Al momento de la publicación, se generará una versión inmutable del producto tributario, que será almacenada en el repositorio de objetos. *Las referencias a objetos* en el almacenamiento de objetos deberán almacenarse en el modelo transaccional únicamente mediante identificadores, URI internas, object keys, checksum, versión u otra metadata necesaria; no deberá replicarse innecesariamente el archivo completo del objeto dentro de Oracle.

Motivación: Reduce duplicidad, evita inconsistencias entre repositorios y mantiene un único origen físico para cada artefacto.

2.3.2. Modelo de componentes internos – ambiente Intranet

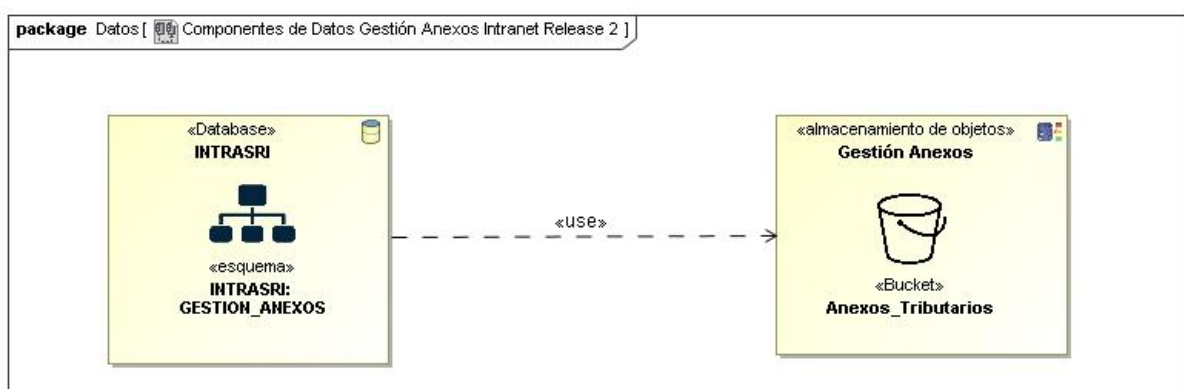


Figura 4: Modelo de Componentes Internos de Datos de la Solución Tecnológica - Entrega y Recepción de Información Fiscal – Gestión Anexos (INTRANET)

Componente	Descripción	Tecnología Utilizada
INTRASRI		
INTRASRI: GESTION_ANEXOS	Descripción: Esquema transaccional del ambiente Intranet que almacena la información estructurada perteneciente al Sistema de Gestión de Anexos - Intranet. Contiene la información administrada por los Bounded Contexts de Estructura, Catálogos, Parametrización/Reglas, Habilitación/Publicación y Autorización, conforme a la responsabilidad de cada componente de dominio. Tipo de información: Datos estructurados y transaccionales. Uso: Persistencia de la configuración y gobernabilidad de Productos Tributarios: estructuras versionadas, catálogos propios del SGA, reglas y validaciones parametrizadas, estados de habilitación/publicación, metadatos de versión y demás información propia.	Esquema de base de datos Oracle versión Anexo A.
Almacenamiento de objetos		
Almacenamiento de Objetos - Bucket Anexo_Tributarios	Descripción: Repositorio de objetos del Sistema Gestión Anexos - Intranet utilizado para almacenar los artefactos generados durante la creación/habilitación de plantillas. Estos objetos materializan la configuración versionada, inmutable y verificable de un Producto Tributario (plantilla). Tipo de información: Objetos semiestructurados. Uso: Almacenamiento de Producto Tributario (plantilla) publicados. Patrones: Object Storage, Versioned Artifact, checksum de integridad.	- MinIO. - Almacenamiento de objetos compatible con S3.

2.3.3. Modelo de componentes internos – ambiente Internet

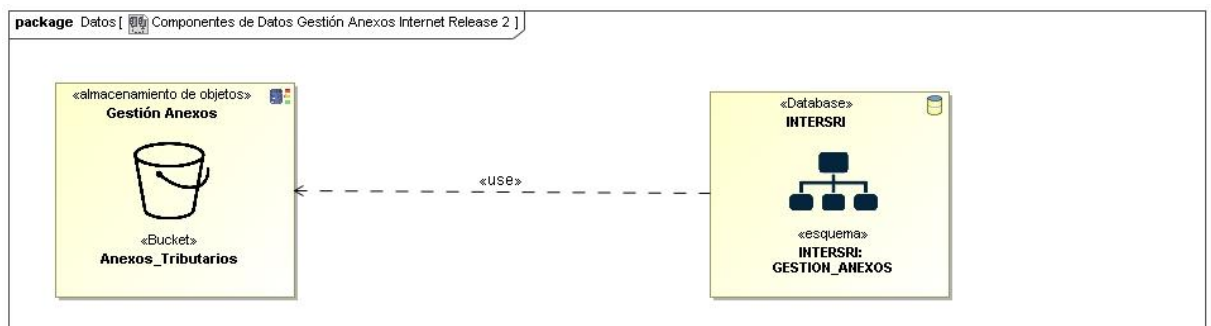


Figura 5: Modelo de Componentes Internos de Datos de la Solución Tecnológica - Entrega y Recepción de Información Fiscal – Gestión Anexos (INTERNET)

Componente	Descripción	Tecnología Utilizada
INTERSRI		
INTERSRI: GESTION_ANEXOS	Descripción: Esquema lógico de aplicación representado dentro de INTERSRI para agrupar los objetos de base de datos del Sistema Gestión Anexos - Internet. Tipo: esquema de datos estructurados. Uso: agrupación de tablas, vistas, secuencias, índices,	Esquema de base de datos Oracle versión Anexo A.

	restricciones y demás objetos necesarios para soportar la operación del ambiente Internet.	
Almacenamiento de objetos		
Almacenamiento de Objetos – Bucket Anexo_Tributarios	Descripción: Repositorio de objetos utilizado por Gestión Anexos para almacenar información no estructurada o semiestructurada asociada al flujo de Internet, como archivos y objetos que no corresponde mantener directamente dentro de la base de datos relacional. En dependencia del flujo se implementará diferente bucket acorde a la necesidad para la recepción y procesamiento. Tipo de información: Objetos semiestructurados. Uso: almacenamiento de archivos/objetos asociados al proceso de anexos. Patrones: Object Storage, Versioned Artifact, checksum de integridad.	• MinIO. • Almacenamiento de objetos compatible con S3.

2.4. Arquitectura de infraestructura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos

2.4.1. Definiciones de Arquitectura

Definición: La aplicación debe ser desplegada utilizando una plataforma de orquestación de contenedores, permitiendo una gestión eficiente, escalabilidad automática y alta disponibilidad del servicio.

Motivación: El uso de una plataforma de contenedores permite mejorar la resiliencia y disponibilidad de la aplicación, facilitando el despliegue continuo y la recuperación ante fallos. Dado que se trata de un servicio crítico, es fundamental garantizar su operatividad constante y su capacidad de escalar según la demanda de los contribuyentes.

Definición: La estrategia de alta disponibilidad y/o escalamiento horizontal a implementar para los componentes de la presente solución será a través del balanceo de instancias que trabajan en modo standalone, dicho balanceo será configurado en el componente F5 y en los servidores apache donde se configura el balanceo.

Motivación: Establecer el mecanismo que debe considerar la solución para el crecimiento horizontal y alta disponibilidad, cuando se lo requiera.

Definición: La solución Gestión Anexos Intranet será desplegada como cargas contenerizadas sobre una plataforma de orquestación Kubernetes, en infraestructura institucional on-premise. La alta disponibilidad y el escalamiento horizontal de los componentes se gestionarán mediante capacidades nativas de Kubernetes, tales como Deployments, ReplicaSets, Services, réplicas de pods, health checks, readiness/liveness probes y políticas de escalamiento, según corresponda.

Motivación: El uso de Kubernetes permite distribuir réplicas de los componentes, recuperar automáticamente pods ante fallos, balancear tráfico dentro del clúster y

escalar horizontalmente según la demanda. Asimismo, la integración con API Management e Istio permite mantener una exposición controlada, segura, trazable y alineada con la arquitectura institucional definida.

Definición: Los componentes legados o institucionales que permanezcan fuera de Kubernetes mantendrán sus mecanismos actuales de despliegue, balanceo y operación, mientras que los componentes nuevos de Anexos deberán registrarse por el modelo contenerizado definido.

Motivación: Permite una convivencia controlada entre la arquitectura vigente y la arquitectura objetivo, evitando forzar migraciones no contempladas dentro del alcance.

Definición: Los componentes de la solución deberán organizarse en namespaces de Kubernetes de acuerdo con criterios de dominio, ambiente, seguridad y operación definidos por la institución.

Motivación: Permite aislar recursos, aplicar políticas diferenciadas, facilitar administración operativa y mejorar el control sobre permisos, cuotas y despliegues.

Definición: Los componentes críticos de la solución deberán ejecutarse con múltiples réplicas de pods, distribuidas en nodos worker cuando la plataforma lo permita, evitando puntos únicos de falla a nivel de aplicación.

Motivación: Garantiza continuidad del servicio ante fallos de pod o nodo, permite mantenimiento controlado y mejora la capacidad de atención concurrente.

Definición: El tráfico administrado dentro del clúster deberá utilizar Istio como Service Mesh cuando corresponda al modelo de plataforma, con Istiod como plano de control y proxies Envoy asociados a los workloads gestionados.

Motivación: Permite aplicar de manera transversal políticas de comunicación, telemetría y seguridad este-oeste sin incorporar dichas responsabilidades en el código de negocio.

2.4.2. Modelo de Despliegue Ambiente Intranet

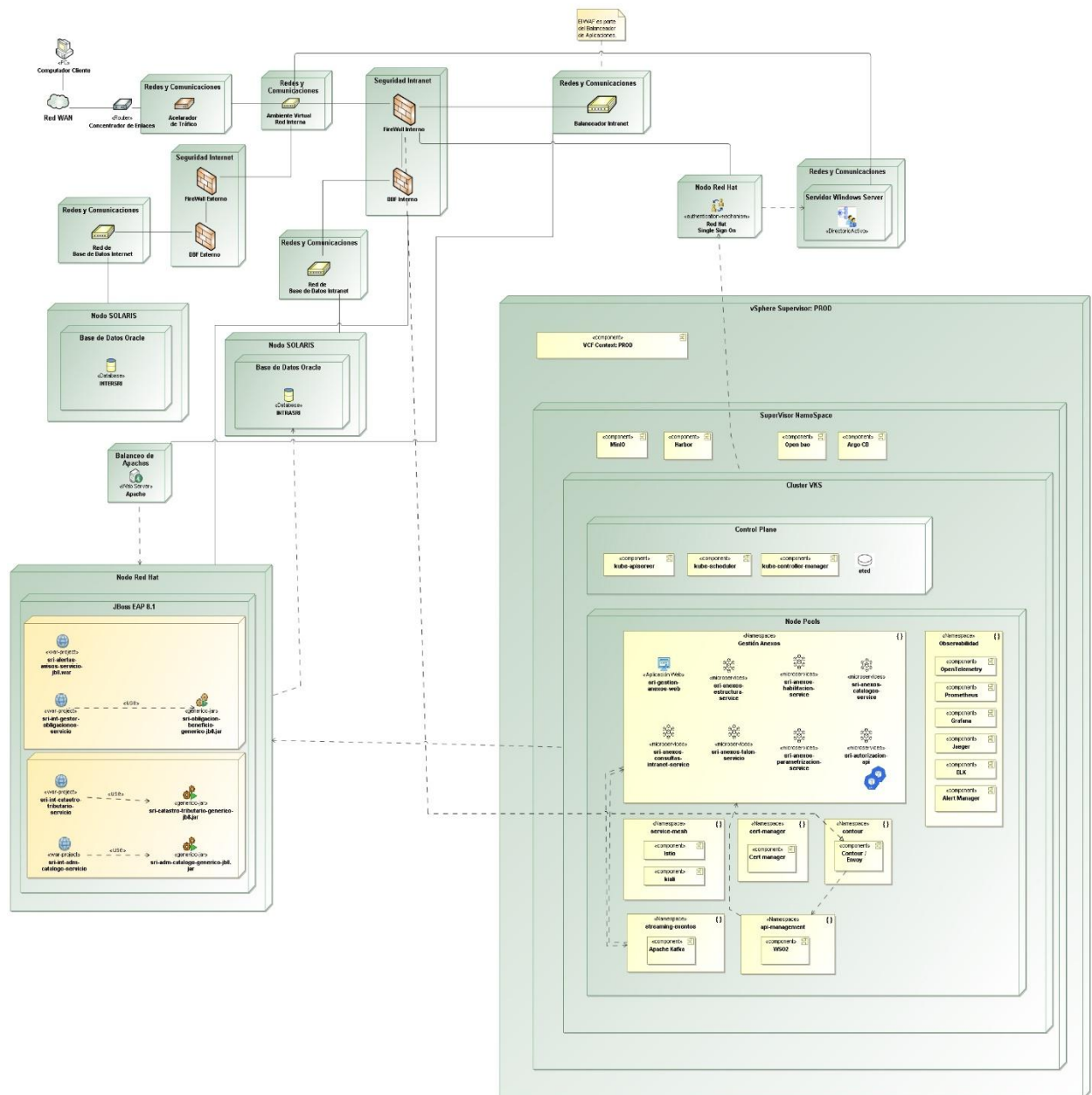


Figura 6: Modelo de Despliegue de la Solución Tecnológica - Entrega y Recepción de Información Fiscal – Gestión Anexos (INTRANET)

Nota:

- Los componentes de Seguridad Internet y Seguridad Intranet se encuentran descritos en la arquitectura general “Arquitectura Tecnológica Seguridad Informática”.
- Los componentes de Redes y Comunicaciones se encuentran descritos en la arquitectura general “Arquitectura Tecnológica Redes y Comunicaciones”.

Equipo	Características
Computador Cliente	Navegador: ver Anexo A

Equipo	Características
Red Hat Single Sign On Internet	Componente que se utiliza para la autenticación del aplicativo. Red Hat Single Sign-On.
INTRASRI	Oracle Database versión Anexo A , instalado en un sistema operativo Solaris. Mediante el firewall de base de datos (DBF) son auditadas las acciones ejecutadas desde la aplicación Anexos y desde cualquier cliente contra esta base de datos.
JBoss EAP	Servidor de Aplicaciones Red Hat JBoss EAP y JDK de acuerdo con lo descrito en Anexo A, instalado en un Red Hat Linux, este servidor de aplicaciones destinado para desplegar los servicios web tipo REST de las integraciones y convivencias. La infraestructura requerida corresponde a 2 servidores RHEL 10.x, cada uno con 2 instancias independientes de JBoss EAP, obteniendo un total de 4 instancias JBoss EAP distribuidas entre ambos servidores.
vSphere Supervisor: PROD	Capa de infraestructura VMware que habilita la ejecución y administración de servicios Kubernetes sobre el entorno de producción. Aloja el Supervisor Namespace desde el cual se aprovisiona y gobierna el clúster VKS.
VCF Context: PROD	Contexto de administración utilizado para operar los recursos de VMware Cloud Foundation en producción. Representa el ámbito de conexión/gestión de VCF.
Supervisor Namespace	Ámbito de vSphere Supervisor que delimita recursos, permisos, políticas de almacenamiento y conectividad para los servicios y clústeres VKS asignados.
Almacenamiento de objetos MinIO	Servicio de almacenamiento de objetos ubicado en el ámbito del Supervisor Namespace, fuera del Cluster VKS representado. Proporciona almacenamiento compatible con S3 para objetos de la solución y requiere persistencia, capacidad y políticas de protección dimensionadas por Infraestructura.
Harbor	Registro de imágenes de contenedor de la plataforma. Aloja las imágenes que serán descargadas por los nodos VKS durante el despliegue de Pods y constituye parte de la cadena de suministro de contenedores.
OpenBao	Servicio de gestión de secretos de plataforma ubicado en el Supervisor Namespace. Centraliza credenciales y material sensible consumido por los workloads sin incorporarlo a las imágenes de contenedor.
Argo CD	Componente GitOps de plataforma desplegado en el ámbito del Supervisor Namespace. Sincroniza manifiestos declarativos desde repositorios versionados hacia el Cluster VKS y mantiene el estado desplegado alineado con la configuración aprobada.
Cluster VKS	Clúster Kubernetes de producción aprovisionado bajo el Supervisor Namespace. Posee API Kubernetes propia, Control Plane y capacidad de cómputo mediante Node Pools para ejecutar los workloads de Gestión Anexos y las herramientas de plataforma mostradas en el diagrama.
Control Plane	Conjunto administrado de componentes Kubernetes responsables del gobierno del clúster. No aloja workloads funcionales de Gestión Anexos.

Equipo	Características
kube-apiserver	Endpoint de la API Kubernetes del clúster. Recibe y valida operaciones sobre recursos Kubernetes y constituye el punto central de comunicación de administradores, controladores y herramientas de automatización.
kube-scheduler	Componente del Control Plane que asigna los Pods a los Worker Nodes disponibles de los Node Pools en función de recursos, restricciones y políticas de scheduling.
kube-controller-manager	Componente del Control Plane que ejecuta los controladores Kubernetes y reconcilia continuamente el estado declarado de Deployments, ReplicaSets, Nodes y demás recursos.
etcd	Almacén distribuido de clave-valor utilizado por el Control Plane para persistir el estado del clúster. Su operación y protección corresponden a la plataforma VKS.
Node Pools	Conjuntos de Worker Nodes del Cluster VKS que aportan CPU, memoria y capacidad de ejecución a los Pods.
Namespace Gestión Anexos	Namespace Kubernetes lógico del Cluster VKS que agrupa los recursos desplegables del aplicativo: Deployments, Pods, Services, ConfigMaps y Secrets.
Namespace Observabilidad	Namespace Kubernetes lógico que agrupa los componentes de telemetría del Cluster VKS. Sus Pods son programados en los Node Pools disponibles.
OpenTelemetry	Collector/servicio de telemetría desplegado en Kubernetes para recibir y exportar trazas, métricas y logs de los workloads instrumentados. Opera como componente de plataforma dentro del namespace Observabilidad.
Prometheus	Servicio de monitoreo desplegado en Kubernetes para recolectar métricas de aplicaciones y plataforma. Requiere recursos de cómputo y almacenamiento persistente acordes con la retención definida por Infraestructura.
Grafana	Servicio de visualización desplegado como workload Kubernetes. Consume las fuentes de métricas/trazas configuradas y se publica mediante los mecanismos de acceso del clúster.
Jaeger	Servicio de trazas distribuido desplegado dentro del namespace Observabilidad. Recibe/consulta información de tracing y requiere capacidad de almacenamiento según la implementación definida.
ELK	Conjunto de componentes Elasticsearch, Logstash y Kibana desplegados para ingestión, persistencia y consulta de logs. Desde infraestructura debe considerarse capacidad de cómputo, almacenamiento persistente y retención.
Alert Manager	Componente de alertamiento asociado a Prometheus, desplegado como workload Kubernetes para agrupar y enrutar alertas operativas.
Namespace service-mesh	Namespace Kubernetes lógico destinado a los componentes del Service Mesh. Se mantiene separado del namespace funcional para administrar de forma independiente la infraestructura de comunicación interna.

Equipo	Características
Istio	Componente de Service Mesh desplegado en el clúster para administrar políticas y telemetría del tráfico entre workloads.
Kiali	Herramienta de observación del Service Mesh desplegada como workload Kubernetes y conectada a las métricas/topología de Istio.
Namespace cert-manager	Namespace Kubernetes reservado para los componentes de administración de certificados del clúster.
Cert Manager	Controlador Kubernetes que automatiza la emisión, renovación y gestión de certificados utilizados por Ingress y workloads, según los emisores configurados en la plataforma.
Namespace contour	Namespace Kubernetes destinado a la infraestructura de Ingress basada en Contour/Envoy.
Contour / Envoy	Controlador/proxy de Ingress desplegado en VKS para recibir y enrutar tráfico HTTP/HTTPS hacia Services Kubernetes. Constituye la entrada técnica al clúster desde las capas de publicación autorizadas.
Namespace streaming-eventos	Namespace Kubernetes lógico destinado a la plataforma de streaming. Agrupa los Pods y recursos necesarios para operar el servicio de eventos.
Streaming de eventos Apache Kafka	Plataforma de streaming desplegada como workloads dentro del clúster. Desde infraestructura requiere múltiples Pods/nodos lógicos, almacenamiento persistente, red interna y dimensionamiento específico de capacidad; los valores físicos quedan sujetos al diseño de plataforma.
Namespace api-management	Namespace Kubernetes lógico reservado para la plataforma de API Management representada en el diagrama final.
API Management — WSO2	Plataforma de API Management desplegada en el Cluster VKS. Se ejecuta como workload de plataforma, se integra con la capa de Ingress y requiere recursos, persistencia/configuración y alta disponibilidad conforme al diseño institucional.

2.4.3. Modelo de Despliegue Ambiente Internet

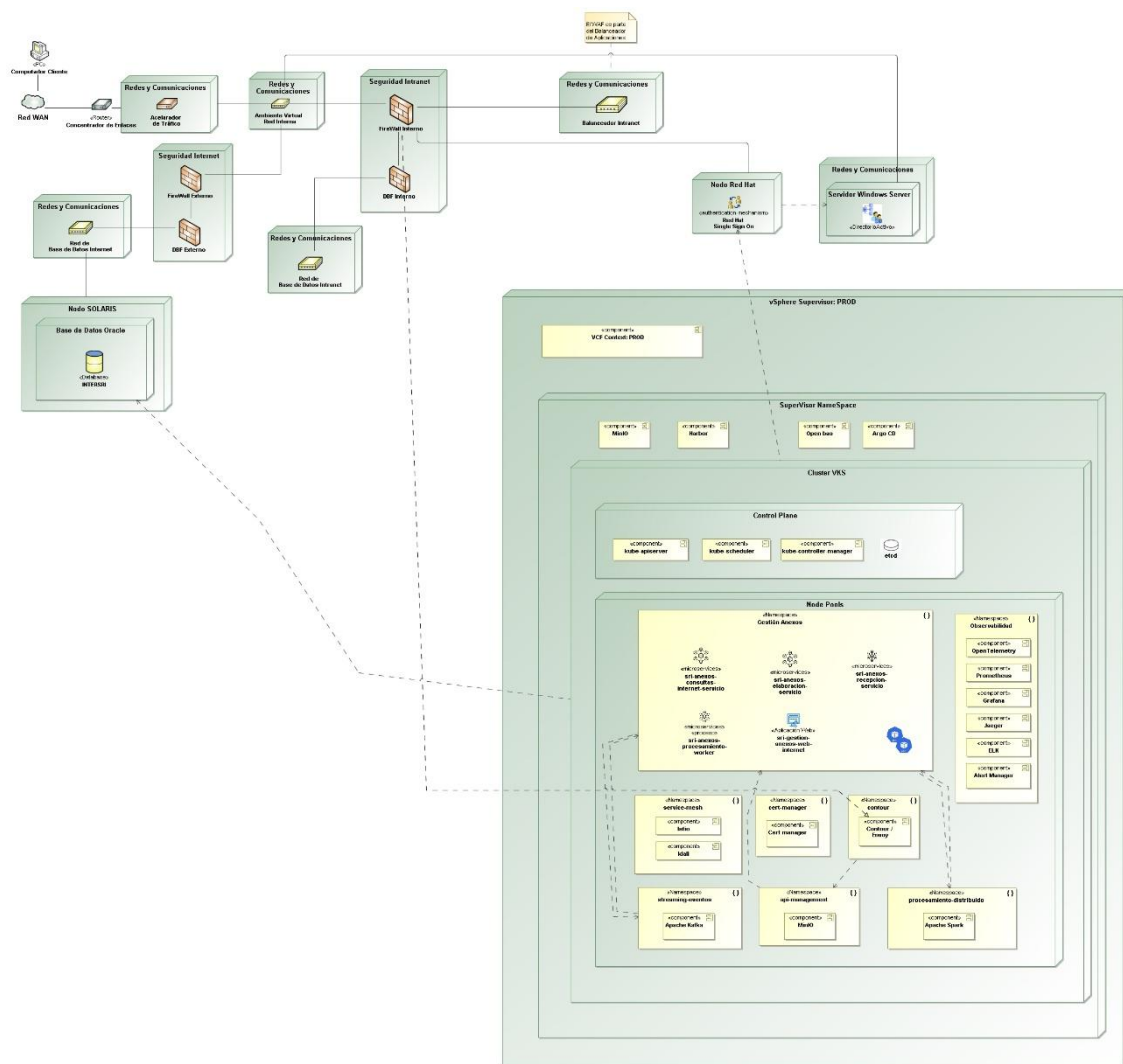


Figura 7: Modelo de Despliegue de la Solución Tecnológica - Entrega y Recepción de Información Fiscal – Gestión Anexos (INTERNET)

Equipo	Características
Computador Cliente	Navegador: ver Anexo A
Red Hat Single Sign On Internet	Componente que se utiliza para la autenticación del aplicativo. Red Hat Single Sign-On.
INTERSRI	Oracle Database versión Anexo A , instalado en un sistema operativo Solaris. Mediante el firewall de base de datos (DBF) son auditadas las acciones ejecutadas desde la aplicación Anexos y desde cualquier cliente contra esta base de datos.
vSphere Supervisor: PROD	Capa de infraestructura VMware que habilita la ejecución y administración de servicios Kubernetes sobre el entorno de producción. Aloja el Supervisor Namespace desde el cual se aprovisiona y gobierna el clúster VKS.
VCF Context: PROD	Contexto de administración utilizado para operar los recursos de VMware Cloud Foundation en producción. Representa el ámbito de conexión/gestión de VCF.

Equipo	Características
Supervisor Namespace	Ámbito de vSphere Supervisor que delimita recursos, permisos, políticas de almacenamiento y conectividad para los servicios y clústeres VKS asignados.
Almacenamiento de objetos	Servicio de almacenamiento de objetos ubicado en el ámbito del Supervisor Namespace, fuera del Cluster VKS representado. Proporciona almacenamiento compatible con S3 para objetos de la solución. .
Harbor	Registro de imágenes de contenedor de la plataforma. Aloja las imágenes que serán descargadas por los nodos VKS durante el despliegue de Pods y constituye parte de la cadena de suministro de contenedores.
OpenBao	Servicio de gestión de secretos de plataforma ubicado en el Supervisor Namespace. Centraliza credenciales y material sensible consumido por los workloads sin incorporarlo a las imágenes de contenedor.
Argo CD	Componente GitOps de plataforma desplegado en el ámbito del Supervisor Namespace. Sincroniza manifiestos declarativos desde repositorios versionados hacia el Cluster VKS y mantiene el estado desplegado alineado con la configuración aprobada.
Cluster VKS	Clúster Kubernetes de producción aprovisionado bajo el Supervisor Namespace. Posee API Kubernetes propia, Control Plane y capacidad de cómputo mediante Node Pools para ejecutar los workloads de Gestión Anexos y las herramientas de plataforma mostradas en el diagrama.
Control Plane	Conjunto administrado de componentes Kubernetes responsables del gobierno del clúster. No aloja workloads funcionales de Gestión Anexos.
kube-apiserver	Endpoint de la API Kubernetes del clúster. Recibe y valida operaciones sobre recursos Kubernetes y constituye el punto central de comunicación de administradores, controladores y herramientas de automatización.
kube-scheduler	Componente del Control Plane que asigna los Pods a los Worker Nodes disponibles de los Node Pools en función de recursos, restricciones y políticas de scheduling.
kube-controller-manager	Componente del Control Plane que ejecuta los controladores Kubernetes y reconcilia continuamente el estado declarado de Deployments, ReplicaSets, Nodes y demás recursos.
etcd	Almacén distribuido de clave-valor utilizado por el Control Plane para persistir el estado del clúster. Su operación y protección corresponden a la plataforma VKS.
Node Pools	Conjuntos de Worker Nodes del Cluster VKS que aportan CPU, memoria y capacidad de ejecución a los Pods.
Namespace Gestión Anexos (INTERNET)	Namespace Kubernetes lógico del Cluster VKS que agrupa los recursos desplegables del aplicativo: Deployments, Pods, Services, ConfigMaps y Secrets.
Namespace Observabilidad	Namespace Kubernetes lógico que agrupa los componentes de telemetría del Cluster VKS. Sus Pods son programados en los Node Pools disponibles.

Equipo	Características
OpenTelemetry	Collector/servicio de telemetría desplegado en Kubernetes para recibir y exportar trazas, métricas y logs de los workloads instrumentados. Opera como componente de plataforma dentro del namespace Observabilidad.
Prometheus	Servicio de monitoreo desplegado en Kubernetes para recolectar métricas de aplicaciones y plataforma. Requiere recursos de cómputo y almacenamiento persistente acordes con la retención definida por Infraestructura.
Grafana	Servicio de visualización desplegado como workload Kubernetes. Consume las fuentes de métricas/trazas configuradas y se publica mediante los mecanismos de acceso del clúster.
Jaeger	Servicio de trazas distribuido desplegado dentro del namespace Observabilidad. Recibe/consulta información de tracing y requiere capacidad de almacenamiento según la implementación definida.
ELK	Conjunto de componentes Elasticsearch, Logstash y Kibana desplegados para ingestión, persistencia y consulta de logs. Desde infraestructura debe considerarse capacidad de cómputo, almacenamiento persistente y retención.
Alert Manager	Componente de alertamiento asociado a Prometheus, desplegado como workload Kubernetes para agrupar y enrutar alertas operativas.
Namespace service-mesh	Namespace Kubernetes lógico destinado a los componentes del Service Mesh. Se mantiene separado del namespace funcional para administrar de forma independiente la infraestructura de comunicación interna.
Istio	Componente de Service Mesh desplegado en el clúster para administrar políticas y telemetría del tráfico entre workloads.
Kiali	Herramienta de observación del Service Mesh desplegada como workload Kubernetes y conectada a las métricas/topología de Istio.
Namespace cert-manager	Namespace Kubernetes reservado para los componentes de administración de certificados del clúster.
Cert Manager	Controlador Kubernetes que automatiza la emisión, renovación y gestión de certificados utilizados por Ingress y workloads, según los emisores configurados en la plataforma.
Namespace contour	Namespace Kubernetes destinado a la infraestructura de Ingress basada en Contour/Envoy.
Contour / Envoy	Controlador/proxy de Ingress desplegado en VKS para recibir y enrutar tráfico HTTP/HTTPS hacia Services Kubernetes. Constituye la entrada técnica al clúster desde las capas de publicación autorizadas.
Namespace streaming-eventos	Namespace Kubernetes lógico destinado a la plataforma de streaming. Agrupa los Pods y recursos necesarios para operar el servicio de eventos.
Streaming de eventos	Plataforma de streaming desplegada como workloads dentro del clúster. Desde infraestructura requiere múltiples Pods/nodos lógicos, almacenamiento persistente, red interna y

Equipo	Características
	dimensionamiento específico de capacidad; los valores físicos quedan sujetos al diseño de plataforma.
Namespace api-management	Namespace Kubernetes lógico reservado para la plataforma de API Management representada en el diagrama final.
API Management — WSO2	Plataforma de API Management desplegada en el Cluster VKS. Se ejecuta como workload de plataforma, se integra con la capa de Ingress y requiere recursos, persistencia/configuración y alta disponibilidad conforme al diseño institucional.
Namespace procesamiento-distribuido	Namespace Kubernetes destinado a la capacidad de procesamiento distribuido. Su existencia lógica permite aislar los recursos de cómputo del motor cuando se habilite.
Motor de procesamiento de datos distribuido	Motor de procesamiento distribuido desplegable sobre VKS únicamente cuando la arquitectura habilite el modo distribuido. Desde infraestructura implica Pods de driver/executors y consumo intensivo de CPU/memoria.

3. Glosario

3.1. Acrónimos

Acrónimo	Significado
ACID	Atomicity, Consistency, Isolation, Durability — Atomicidad, Consistencia, Aislamiento y Durabilidad
API	Application Programming Interface — Interfaz de Programación de Aplicaciones
API Mgmt	API Management — Gestión de APIs
AST	Abstract Syntax Tree — Árbol de Sintaxis Abstracta
BLOB	Binary Large Object
BRMS	Business Rules Management System — Sistema de Gestión de Reglas de Negocio
CDI	Contexts and Dependency Injection
CI/CD	Continuous Integration / Continuous Delivery o Deployment — Integración y Entrega/Despliegue Continuo
CLOB	Character Large Object
CPU	Central Processing Unit — Unidad Central de Procesamiento
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design — Diseño Dirigido por el Dominio
DLQ	Dead Letter Queue
DPT	Declaración Patrimonial; el documento utiliza la denominación “Declaración Patrimonial Tributaria”
DSL	Domain-Specific Language — Lenguaje Específico de Dominio
EAP	Enterprise Application Platform
EDA	Event-Driven Architecture — Arquitectura Orientada a Eventos
EFK	Elasticsearch, Fluentd/Fluent Bit, Kibana
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure

JAR	Java Archive
JDK	Java Development Kit
JSON	JavaScript Object Notation
JSON-B	Jakarta JSON Binding
MFA	Multi-Factor Authentication — Autenticación Multifactor
NPM	Node Package Manager
OAuth2	OAuth 2.0 — protocolo/framework de autorización
OIDC	OpenID Connect
PMD	Herramienta de análisis estático utilizada por el estándar institucional.
R1	Release 1
R2	Release 2
REST	Representational State Transfer
RHEL	Red Hat Enterprise Linux
RUC	Registro Único de Contribuyentes
S3	Simple Storage Service; interfaz/protocolo de almacenamiento de objetos compatible con Amazon S3
SGA	Sistema de Gestión de Anexos
SOAP	Simple Object Access Protocol
SPA	Single Page Application
SQL	Structured Query Language
SRI	Servicio de Rentas Internas
SSO	Single Sign-On
TLS	Transport Layer Security
UML	Unified Modeling Language
URI	Uniform Resource Identifier
VKS	vSphere Kubernetes Service
WAF	Web Application Firewall

3.2. Definiciones

Término	Definición propuesta
ACID	Conjunto de propiedades de una transacción de base de datos: Atomicidad, Consistencia, Aislamiento y Durabilidad, orientadas a garantizar integridad y confiabilidad transaccional.
API Server / kube-apiserver	Componente del Control Plane de Kubernetes que expone la API del clúster y constituye el punto de entrada para consultar o modificar el estado de los recursos.
Control Plane	Conjunto de componentes de Kubernetes responsables de gobernar el clúster, mantener su estado deseado, exponer su API, programar cargas y persistir la configuración del clúster.
Data Plane	Conjunto de nodos y componentes donde se ejecutan las cargas de trabajo y se procesa el tráfico de las aplicaciones.

etcd	Almacén distribuido clave-valor utilizado por Kubernetes para conservar de manera consistente el estado y configuración del clúster.
Event-Driven Architecture — EDA	Estilo arquitectónico en el que productores emiten eventos y los consumidores reaccionan a ellos de forma desacoplada, favoreciendo asincronía, escalabilidad y resiliencia.
Kube Scheduler / kube-scheduler	Componente del Control Plane que selecciona el nodo más adecuado para ejecutar cada Pod considerando recursos, restricciones y políticas de scheduling.
Kubelet	Agente que se ejecuta en cada worker node y garantiza que los contenedores especificados para los Pods se encuentren ejecutándose conforme al estado deseado.
kube-proxy	Componente de red de Kubernetes que implementa reglas necesarias para el acceso a Services y el encaminamiento del tráfico hacia sus endpoints.
Kubernetes	Plataforma de orquestación de contenedores que automatiza despliegue, operación, escalamiento y recuperación de aplicaciones contenerizadas.
Microservicios	Enfoque arquitectónico que descompone una solución en servicios autónomos, desplegables independientemente y alineados a capacidades o límites funcionales bien definidos.
Namespace	Mecanismo lógico de Kubernetes utilizado para agrupar y aislar recursos dentro de un mismo clúster, facilitando organización, políticas, permisos y administración de recursos.
Node / Nodo	Máquina física o virtual integrante de un clúster Kubernetes. Un worker node proporciona capacidad de cómputo para ejecutar Pods.
OpenAPI	Especificación estándar para describir y documentar APIs HTTP/REST, incluyendo rutas, operaciones, parámetros, modelos de datos, respuestas y contratos de interoperabilidad.
Pod	Unidad desplegable mínima de Kubernetes.
Quarkus	Framework Java orientado a aplicaciones cloud-native y microservicios, optimizado para contenedores, tiempos de arranque reducidos y consumo eficiente de recursos.
REST	Estilo arquitectónico para sistemas distribuidos basado en recursos, comunicación sin estado, interfaz uniforme y uso de las capacidades semánticas de HTTP.

Service Mesh	Capa de infraestructura que gestiona la comunicación servicio-a-servicio y habilita capacidades transversales como seguridad, políticas de tráfico, resiliencia, telemetría, balanceo y trazabilidad sin incorporarlas directamente en la lógica de negocio.
Vue.js	Framework progresivo de JavaScript utilizado para construir interfaces web basadas en componentes y comportamiento reactivo.

4. Bibliografía

The New Stack. (n.d.). How Redis simplifies microservices design patterns. The New Stack.
<https://thenewstack.io/how-redis-simplifies-microservices-design-patterns/>

Fowler, M. (2011, 14 de julio). CQRS. Martin Fowler's Bliki. <https://martinfowler.com/bliki/CQRS.html>

Newman, C. (n.d.). Saga pattern. Microservices.io. Retrieved February 21, 2025, from
<https://microservices.io/patterns/data/saga.html>

XenonStack. (2024, December 18). Microservices Architecture and Design Patterns. Retrieved from
<https://www.xenonstack.com/insights/microservices>

Matei, Z., & Xin, R. (2022). Apache Spark: Unified Engine for Big Data Processing. Apache Software Foundation. <https://spark.apache.org/>

Oracle Corporation. (2023). Oracle Database JSON Developer's Guide.
<https://docs.oracle.com/en/database/oracle/oracle-database/21/adjsn/>

Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley Professional.

Broadcom Inc. (2026). vSphere Kubernetes Service architecture and components. Broadcom TechDocs.
<https://techdocs.broadcom.com/us/en/vmware-cis/vcf/vcf-9-0-and-later/9-1/vsphere-supervisor-installation-and-configuration/vsphere-supervisor-concepts/tanzu-kubernetes-grid-architecture-and-components.html>

5. Anexos

5.1. Anexo A: Versión de tecnologías

Componente	Versión
BackEnd	
JDK	Red Hat OpenJDK 21.0.8. con su última actualización y/o parche liberado.
Quarkus	3.36.1
Lombok	1.18.42
MapStruct	1.6.3.redhat-00001
FrontEnd	

Vue	3.5.24
Node	22.19.0
NPM	10.8.3
Vue Cli	5.0.8
TypeScript	5.9.3
Vite	7.3.2
PrimeVue	4.3.7
Primeicons	7.0.0
BootStrap	5.3.7
Pinia	3.0.4
Axios	1.16.0
Keycloak-js	26.2.0
formulajs	4.6.0
vuelidate/core	2.0.3
vuelidate/validators	2.0.4
pawel-up/jexl	4.4.2
uuid	14.0.0
chevrotain	12.0.0
vue-codemirror6	1.5.1
vuedraggable	4.0.1
read-excel-file	9.0.9
vue-i18n	11.4.5
vue-router	4.6.4
v-idle-3	0.3.14
@primeuix/themes	2.0.3
jwt-decode	4.0.0
@fortawesome/fontawesome-svg-core	7.2.0
@fortawesome/free-brands-svg-icons	7.2.0
@fortawesome/free-regular-svg-icons	7.2.0
@fortawesome/free-solid-svg-icons	7.2.0
@fortawesome/vue-fontawesome	3.2.0
@codemirror/lang-json	6.0.2
@codemirror/language	6.12.3
@codemirror/theme-one-dark	6.1.3
@codemirror/view	6.41.1
Especificación Jakarta EE 10	
JDK	Red Hat OpenJDK 21.0.8. con su última actualización y/o parche liberado.
JBoss	EAP 8.1.0. con su última actualización y/o parche liberado.
Jakarta Persistence	3.1

Jakarta Contexts and Dependency Injection	4.0
Jakarta Servlet	6.0
Jakarta Enterprise Beans	4.0
Jakarta Bean Validation	3.0
Jackson	2.17
Jakarta Interceptors	2.1
Jakarta Transactions	2.0
Jakarta Restful Web Services	JAX-RS 3.1
Base de datos	
INTRASRI	Oracle 19c Enterprise Edition o superior
INTERSRI	Oracle 19c Enterprise Edition o superior
Red Hat Single Sign On	7.1.4.GA
Linux	Red Hat 10.x con su última actualización y/o parche liberado
Navegadores	• Mozilla Firefox 146.0 o Superior • Google Chrome 110 o Superior • Microsoft Edge 143.0 o Superior

5.2. Anexo B: Matriz de integraciones y convivencias

La matriz se encuentra en el confluence en la siguiente ubicación:

[Matriz de Integraciones y Convivencias Anexos](#)

5.3. Anexo C: Estructura librería pistas-auditoria-lib v1.0.0

Figura que define la estructura e información del componente JAR de la librería de pistas de auditoría.

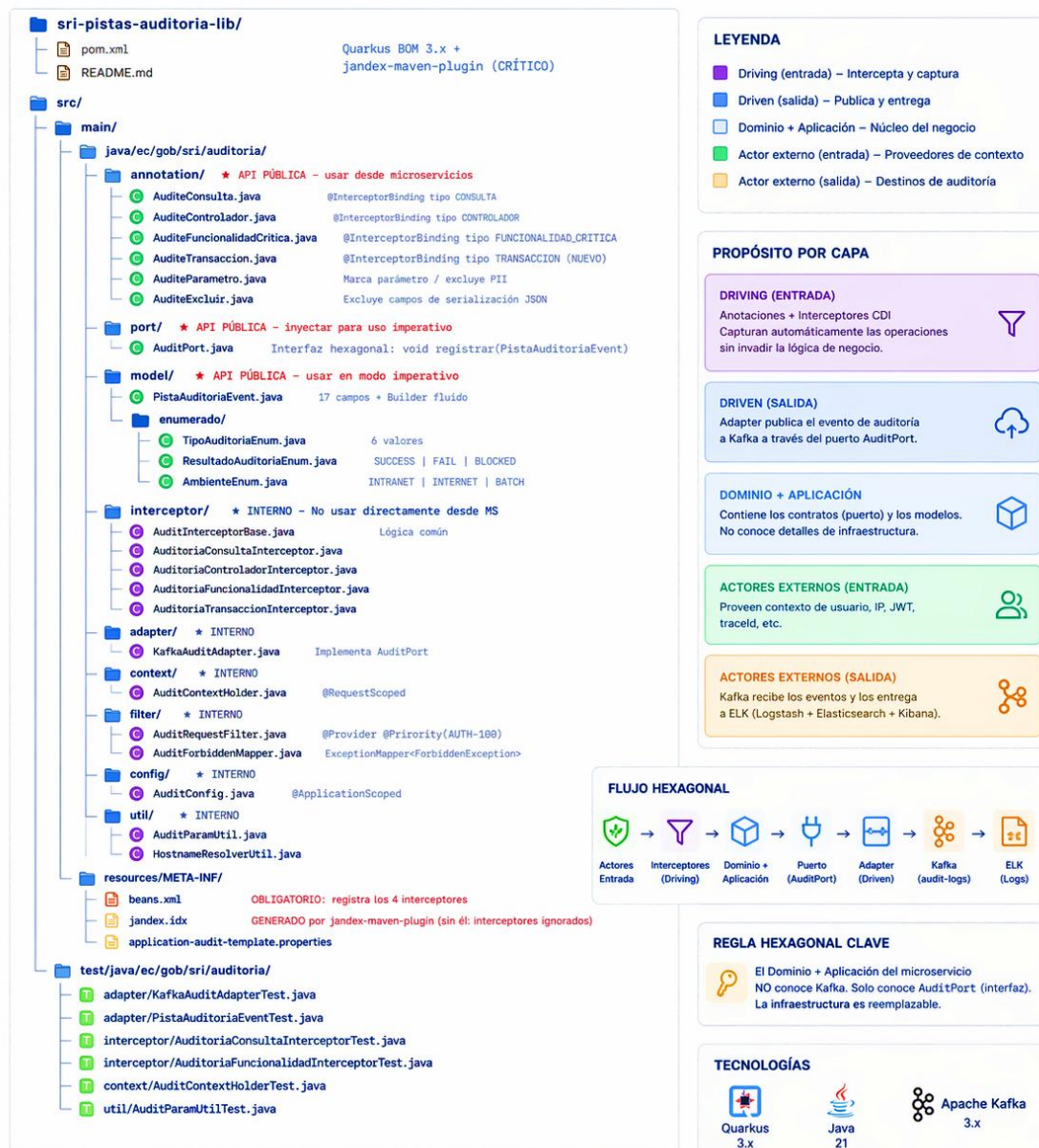


Figura 8. Estructura de librería pistas de auditoria

Responsabilidades por paquete

PAQUETE	TIPO	RESPONSABILIDAD	VISIBILIDAD DESDE MS CONSUMIDOR
annotation/	API PÚBLICA	5 anotaciones @InterceptorBinding. El desarrollador las aplica en Application Services	Usar directamente
port/	API PÚBLICA	AuditPort — contrato hexagonal. Inyectar para uso imperativo opcional	Inyectar si necesario
model/	API PÚBLICA	PistaAuditoriaEvent + 3 enumerados. Usar en modo imperativo con AuditPort	Usar en modo imperativo
interceptor/	INTERNO	4 interceptores CDI @AroundInvoke. Construyen el evento, instrumentan Uni, publican	NO usar directamente

adapter/	INTERNO	KafkaAuditAdapter implementa AuditPort. Emitter Kafka acks=all. Absorbe errores	NO usar directamente
context/	INTERNO	AuditContextHolder @RequestScoped. Aisla IP/hostname/origen por request	NO usar directamente
filter/	INTERNO	AuditRequestFilter captura headers HTTP. AuditForbiddenMapper mapea 403 a pista	NO usar directamente
config/	INTERNO	AuditConfig lee @ConfigProperty al startup. Valida campos obligatorios	NO usar directamente
util/	INTERNO	AuditParametroUtil serializa parámetros. HostnameResolverUtil resuelve hostname async	NO usar directamente

Modelo del Evento PistaAuditoria - JSON

JSON publicado al topic de Kafka.

```
{
  "usuario": "jada270709",
  "ipCliente": "10.12.1.50",
  "hostname": "pc-jada270709-01",
  "fechaHora": "2026-04-08T14:30:00.123",
  "codigoAplicacion": "GAN-PLANTILLA",
  "codigoModulo": "GAN",
  "tipoAuditoria": "FUNCIONALIDAD_CRITICA",
  "ambiente": "INTRANET",
  "codigoAplicacionOrigen": "GAN-PLANTILLA",
  "accion": "APROBAR_PLANTILLA",
  "traceld": "4bf92f3577b34da6a3ce929d0e0e4736",
  "resultado": "SUCCESS",
  "recurso": "GanPlantilla",
  "recursold": "12345",
  "mensajeError": null,
  "versionServicio": "1.0.0",
  "nombrePod": "sri-anexos-estructura-service-jvm-7d9f-xk2"
}
```